



Lukas Lamster, BSc

DRAM Integrity Protection with Efficient Low-Latency Cryptographic Primitives

MASTER'S THESIS

to achieve the university degree of

Diplom-Ingenieur

Master's degree programme: Information and Computer Engineering

submitted to

Graz University of Technology

Supervisors

Daniel Gruss

Maria Eichlseder

Institute of Applied Information Processing and Communications

Graz, October 2021

AFFIDAVIT

I declare that I have authored this thesis independently, that I have not used other than the declared sources/resources, and that I have explicitly indicated all material which has been quoted either literally or by content from the sources used. The text document uploaded to TUGRAZonline is identical to the present master's thesis.

Date, Signature

Acknowledgements

First, I want to thank my advisors, Daniel Gruss and Maria Eichlseder, for making this thesis possible. Their support and feedback were a significant help for me and strongly shaped the outcome of this work. I cannot think of a better advisor duo, and I look forward to working with them in the future.

Furthermore, I want to thank Jonas Juffinger and Andreas Kogler for the insightful discussions that we had. These collaborative brainstorming sessions were an excellent opportunity to work out some of the more complicated details of our mitigation.

I want to thank my parents, Anton and Elisabeth, and my sister, Elina. Thanks to them, I always look forward to coming home, even if it's only for the weekend. They supported me through my entire life and were always there for me. Needless to say, I would not be where I am today without them backing me up. Also, thanks for the delicious cooking.

As I highly appreciate a healthy work-life balance, I must thank my friends for helping me to take my mind off of work. They never fail to make me laugh and brighten up my mood.

Finally, I want to thank Lea for keeping me sane during stressful times. She made the COVID lockdowns feel like a vacation. Thanks to her, I always have someone to talk to. Thank you for putting up with me and for supporting me all this time.

Abstract

Computers are the most pervasive technological devices and are used by billions of people every day. Thus, it is vital to guarantee that the processed data is kept safe and cannot be corrupted. A large share of computing devices uses DRAM to store data currently processed by the system. Unfortunately, research shows that DRAM chips are inherently prone to errors. While environmental disturbances like cosmic radiation might cause system instability, maliciously induced bit errors pose a much more significant threat to system security. A hardware bug named *Rowhammer* allows an attacker to induce single- and multi-bit errors through unprivileged software. Over the past few years, various countermeasures claimed to mitigate Rowhammer and the exploits that come with it. Recent studies show that these mitigations are insufficient and that a sophisticated attacker can circumvent them. Commodity integrity protection schemes like ECC cannot protect against Rowhammer, as attacks bypass the error correction and cause silent data corruption. This work proposes a novel Rowhammer mitigation that simultaneously acts as an integrity protection scheme for commodity DRAM modules starting from DDR4. Building upon the idea of ECC memory, we protect DRAM data with non-linear MAC functions. During system startup, our mitigation generates a random key. Using cryptographic primitives allows our mitigation not only to protect against Rowhammer, but we also can give precise security margins as the attacker needs to break a cryptographic primitive. Unlike ECC, we do not perform error correction in hardware. Instead, we shift this step to the Linux kernel as software-based correction allows for dynamic and adaptive error correction while increasing the number of correctable bits.

To show the practicability of our approach, we model our hardware-software co-design in the gem5 simulator and the Linux kernel. With this simulator, we benchmark our implementation under varying workloads and bitflip patterns. Additionally, we evaluate the security of our design mathematically based on the security of the underlying cryptographic primitives.

Our results prove that our proposed mitigation effectively protects against Rowhammer attacks and naturally occurring bit errors with sufficiently large security margins. We show that software-based error correction offers much greater flexibility than commodity systems and paves the way for new, more sophisticated approaches. Through simulation, we demonstrate that during error-free operation, our mitigation outperforms commodity ECC solutions for certain workloads.

Keywords: Rowhammer Mitigation, Integrity Protection, DRAM Integrity, Rowhammer

Kurzfassung

Computer sind allgegenwärtige Geräte, die täglich von Millionen von Menschen benutzt werden. Daher ist es essentiell, die Sicherheit und Integrität der verarbeiteten Daten zu gewährleisten. Ein großer Teil aller Computer benutzt DRAM als Datenspeicher für Daten, die momentan verarbeitet werden. Unglücklicherweise zeigen Studien, dass die Mikrochips, die in DRAM verwendet werden, fehleranfällig sind. Während natürliche Störquellen, wie etwa kosmische Strahlung, die Systemstabilität gefährden können, stellen böswillig verursachte Fehler eine große Gefahr für die Systemsicherheit dar. Ein Hardware-Fehler namens *Rowhammer* ermöglicht es einem Angreifer, in einem oder mehreren Bit, nur durch unprivilegierte Software, Fehler zu induzieren. In den letzten Jahren wurden zahlreiche Schutzmaßnahmen gegen Rowhammer vorgestellt. Aktuelle Versuche zeigen aber, dass diese Schutzmaßnahmen nicht ausreichend gegen Rowhammer schützen, da sie umgangen werden können. Auch aktuelle Integritätsschutzmaßnahmen wie etwa ECC bieten keinen ausreichenden Schutz gegenüber Rowhammer, da ein Angreifer die Fehlererkennung umgehen kann.

In dieser Arbeit präsentieren wir eine neuartige Schutzmaßnahme gegen Rowhammer, die gleichzeitig die Integrität von Speichermodulen garantiert. Wir schützen die Daten im DRAM mit kryptographischen MACs, deren Schlüssel bei jedem Systemstart neu generiert wird. Da unsere Schutzmaßnahme auf einem kryptographischen Primitiv aufbaut, können wir exakte Sicherheitsschranken angeben. Im Gegensatz zu ECC erfolgt die Fehlerkorrektur bei unserer Methode in Software. Wir verschieben die Korrektur in den Linux Kernel. Dadurch erhöhen wir die Anzahl der korrigierbaren Bits und erhöhen die Flexibilität der Fehlerkorrektur.

Um die Machbarkeit unseres Ansatzes zu untermauern, implementieren wir unser System im gem5 Simulator und die Fehlerkorrektur in einem modifizierten Linux Kernel. Mit dieser Implementierung messen wir die Performance unserer Lösung unter einer Reihe von Arbeitspaketen und in verschiedenen Bitflip-Szenarien. Zusätzlich evaluieren wir die Sicherheit unseres Designs in Form einer mathematischen Analyse, basierend auf dem zugrundeliegenden kryptographischen Primitiv.

Aus unseren Resultaten geht hervor, dass unsere Lösung effektiv gegen Rowhammer schützt. Wir zeigen somit, dass software-basierte Fehlerkorrektur viel mehr Flexibilität als aktuelle ECC-basierte Systeme bietet. Damit ebnen wir den Weg für fortschrittlichere Ansätze. Durch unsere Simulation zeigen wir, dass unsere Schutzmaßnahme um ein Vielfaches weniger Overhead generiert, als klassische ECC Implementierungen.

Schlagwörter: Rowhammer Schutzmaßnahme, Integritätsschutz, DRAM Integrität, Rowhammer

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Contributions	2
1.3	Outline	3
2	Background	5
2.1	DRAM Organization	5
2.2	DRAM Faults, Rowhammer Mitigations and Integrity Protection	11
2.3	Intel x86-64 CPU Architecture	22
2.4	Interrupt and Exception Handling in the Linux Kernel	27
2.5	System Emulation with gem5	30
2.6	Cryptographic Hash Functions and MACs	35
3	Design Approach and High-Level Overview	39
3.1	Threat Model	39
3.2	General Overview	40
3.3	Protecting Integrity with MACs	42
3.4	Error Correction in Software	44
4	Hardware Modifications	47
4.1	Modifying the Memory Controller	47
4.2	CPU Modifications	49
5	Software Modifications	51
5.1	Introducing a new Fault Type	51
5.2	Error Correction in the Linux Kernel	52
6	Evaluation	55
6.1	Evaluation Setup	55
6.2	Security Margins	56
6.3	Time Complexity of the Error Correction	61
6.4	Performance	63
7	Conclusion	67
	Bibliography	69

Chapter 1

Introduction

DRAM modules are a vital component in modern computers as they store data needed by the operating system and user processes likewise. When using DRAM, we implicitly assume that stored data is only modified by write operations of authorized processes. In 2014, Kim et al. [Kim+14] demonstrated that this assumption is false. They showed that an unprivileged process can alter the contents of memory cells by accessing nearby memory cells in quick succession. This effect is known as *Rowhammer* and is the first known hardware fault attack that can be triggered from unprivileged software. Seaborn et al. [SD15] used Rowhammer to escape the NaCl-Sandbox of the Chrome web browser and also mounted a privilege escalation attack with the induced bitflips. Thus, they demonstrated that Rowhammer poses a significant threat to system security.

Following the initial discovery of Rowhammer, researchers published a multitude of exploits based on the same principle as the original Rowhammer paper [AAA17; GMM16; BM16; Rid+21; Wei+19; Zha+20]. In these follow-up papers, researchers enhance the initial attack in various aspects. With Rowhammer.js, Gruss et al. [GMM16] trigger bitflips through malicious JavaScript code, thus demonstrating that Rowhammer attacks are mountable through web browsers. In 2018, Lipp et al. [Lip+20] and Tatar et al. [Tat+18b] both published purely remote Rowhammer attacks that can trigger bitflips on a target machine over a network. These findings significantly increase the attack surface of Rowhammer, as such attacks can target many machines connected to the internet.

1.1 Motivation

As a reaction to these attacks, researchers proposed multiple countermeasures aimed at mitigating Rowhammer or its repercussions. Seeing how Rowhammer poses a threat to system security, CPU vendors have already implemented some of the proposed countermeasures. The mitigations mainly differ in the approach they take to prevent Rowhammer attacks and can be classified into three categories [Gru+18]. Detection-based mitigations detect suspicious access patterns and may prevent code that triggers bitflips from executing. Elimination-based approaches aim to decrease the probability of Rowhammer-induced bitflips, usually by performing additional refresh operations or decreasing the access speed of suspected attacker processes. Other mitigations allow bitflips to happen but try to neutralize the effects of such flips on system security by physically isolating memory rows.

Unfortunately, none of the mitigations proposed so far can reliably protect against sophisticated Rowhammer attacks [GMM16; Sal21; Lip+20; AAA17; Coj+19; Rid+21; Gru+18; Fri+20].

CPUs that can use ECC memory perform integrity checks at each data access. On integrity violation, ECC solutions correct the corrupted data, if possible. Error correction is possible by adding additional redundancy in the form of checksums on the DRAM modules. Most implementations can correct single-bit errors and detect up to double-bit faults based on the implemented ECC algorithm. Due to these error correction and detection capabilities, ECC was thought to be immune to Rowhammer attacks.

In 2019, Cojocar et al. [Coj+19] bypassed ECC error detection with sophisticated bitflip patterns. They managed to induce uncorrectable and also undetectable errors in commodity ECC memory. Their attack finds that the correction mechanism can even worsen the impact of bitflips, as it induces silent data corruption by correcting the memory to a false value.

We want to address these shortcomings by introducing a new correction-based mitigation that combines the efficiency of ECC-based error correction with cryptographic-strength protection against Rowhammer.

1.2 Contributions

This thesis introduces a novel Rowhammer mitigation that simultaneously serves as integrity protection for DRAM modules. Our hardware-software co-design protects against Rowhammer with cryptographic security margins. To show the practicability of our proposed mitigation, we create a reference implementation using the gem5 simulator [Bin+11] and a modified Linux kernel. We test the performance of our countermeasure by running benchmarks on the simulated modified system. Comparing these benchmarks with results acquired on an unmodified system allows us to evaluate the performance overhead due to our mitigation. Furthermore, we derive security margins based on the underlying cryptographic primitive used by the mitigation. With these margins, we show that an attacker will never be able to induce exploitable bitflips in realistic scenarios if our mitigation is active.

The main contributions of this work are as follows.

- We introduce a new Rowhammer countermeasure that protects DRAM data with cryptographic MACs. Our mitigation offers cryptographic-strength protection against Rowhammer attacks. Our results show that an attack on our mitigation is not feasible.
- We introduce modifications to the x86-64 ISA. A new fault type, triggered by the mitigation, signals the checksum mismatches detected by the memory controller. We allow for fast error correction in software with a novel instruction that extends the AVX512 instruction set.
- As we shift the correction to software, we modify the Linux kernel to react to the new fault type and correct corrupted data. We implement a simplified version of

the algorithm described in this thesis as a proof of concept. In the correction, we use the novel instruction that performs quick MAC computations in hardware.

- We provide a reference implementation as a benchmark system. We implement the proposed hardware changes in the gem5 simulator and extend the implementation of the X68-64 ISA with our novel instruction.
- Through benchmarks, we show that our mitigation has negligible impact on system performance and is thus highly suitable even for upcoming DRAM generations. We show that, in most cases, our mitigation performs better than commodity ECC in terms of speed by a factor of approximately 7.

1.3 Outline

The remainder of this thesis is structured as follows. Chapter 2 discusses DRAM organization, DRAM faults, Rowhammer mitigations, and integrity protection schemes. Furthermore, it discusses Intel’s implementation of the x86-64 CPU architecture, the basics of cryptographic hash functions and MACs, how the Linux kernel handles faults and interrupts, and, finally, how the gem5 simulator simulates computer systems with high accuracy.

Chapter 3 discusses the threat model and gives an overview of the mitigation. We elaborate on how DRAM data can be protected with cryptographic primitives and address the issue of software-based error correction.

In Chapter 4, the necessary hardware modifications in the CPU and the memory controller are discussed. We also elaborate on the implementation of these changes in the gem5 simulator.

Chapter 5 gives a detailed description of the software modifications that we introduce in the Linux kernel.

We evaluate the mitigation both mathematically and empirically in Chapter 6. We derive the security margins and show that our mitigation has a negligible impact on the performance during benchmark runs.

Finally, Chapter 7 summarizes the results.

Chapter 2

Background

In this chapter, we provide an in-depth introduction on the topics fundamental for this thesis. Section 2.1 covers the internal structure and workings of DRAM components. In Section 2.2, we address possible faults in DRAM chips and their sources. Additionally, we discuss the mitigation of these faults and how state-of-the-art DRAM components provide data integrity. Section 2.3 provides information on Intel’s x86-64 CPU architecture, focusing on the memory controller and the AVX512 instruction set. An introduction to cryptographic hash functions and their security properties is given in Section 2.6. Interrupt handling in the Linux Kernel is covered in Section 2.4, where we also discuss different interrupt types and page fault handling. Section 2.5 addresses the gem5 simulator, its memory subsystem, and the different CPU and system emulation modes.

As the following sections will often refer to the Linux kernel and the gem5 simulator, note that all references to the Linux kernel refer to kernel version Linux/x86 5.9.2, and all references to gem5 refer to version 21.0.0.0. Statements about the Linux kernel always refer to the unmodified kernel unless stated otherwise.

2.1 DRAM Organization

Many modern DDR4 memory modules are implemented as dual in-line memory modules (DIMMs) or small outline dual in-line memory modules (SO-DIMMs) in the case of mobile devices. These memory modules or *DRAM modules* are the physical components that connect to the memory controller via so-called *channels*. Depending on the used CPU, multiple channels might be available. While DIMMs in different channels can operate in parallel, operations on DIMMs in the same channel have to be time-multiplexed. Using multiple channels increases the effective data throughput by serving requests concurrently. Modern CPUs that support multi-channel modes do not require manual configuration. Instead, the system detects the correct channel configuration at boot time. For appropriate configurations, continuous memory regions can be spread over multiple DIMMs in different channels to allow parallel operation. This feature is called *memory interleaving*, and it aims to increase the average throughput [Int17].

The memory controller communicates with the DIMM via the *data-*, *control-*, *address-* and *chip-select bus* [JWN10]. All data transferred between the DRAM chips and the memory controller runs over the data bus. Depending on the type of memory, the bus width is 64 or 72 bit. The control bus carries signals that instruct the DRAM chips

to perform low-level operations. During memory accesses, the values asserted on the chip-select bus and the memory bus specify the physical location that data is written to or read from.

Each side of a DIMM accommodates several DRAM chips. In DDR4 modules, these chips have data widths of 4, 8, or 16 bit. To achieve the required data bus width of 64 or 72 bit, several DRAM chips are grouped to form a *rank*. Grouping several chips to a *rank* increases the cumulative data bus width to the required 64 or 72 bit. All chips in a rank operate synchronously on the same control signals as they are all connected to the same chip-select bus. Depending on the bus size of the chips, one DIMM can have up to eight ranks. The ranks share the same address-, data- and control bus but have distinct chip-selects. As chips in a rank operate synchronously, a rank can be abstracted as a single chip with 64 or 72-bit data bus width.

Ranks are further subdivided into *banks* and *rows*. A bank consists of a set of *sense amplifiers*, also referred to as the *row buffer*, and multiple rows. On DDR4 modules, banks are grouped into *bank groups*, each group comprises four banks. Four bank groups form a complete rank. Thus, a rank consists of 16 banks overall [JED17]. Banks can operate in parallel, as they are independent of each other.

Each row in a bank is composed of so-called *memory cells*. These cells are the most fundamental component in the memory hierarchy. A single cell holds precisely one bit of information represented by an electrical charge. The JEDEC standard defines the exact voltages of these charges. DDR4 modules use a peak voltage of 1.2 V [JED17]. *Word lines* connect all cells of a single row. Cells at equal offsets in the rows share a common *bit line*. Bit lines connect all cells in a *column*.

As can be seen in Figure 2.1, a memory cell consists of a transistor and a capacitor. By storing the charge, the capacitor holds the information of the cell. Its low capacity allows for quick modification of the charge. The gate of the transistor is connected to the word line. It acts like a switch and connects the capacitor to the bit line.

Each bit line connects to a sense amplifier. These amplifiers are in turn connected to a demultiplexer which is commonly called the *column decoder*. It is impossible to read from or write to a single memory cell exclusively in a DRAM chip. Instead, these operations are always performed on a memory row. With the column decoder, it is possible to route a subset of the sense amplifiers to the data bus. The chip bus size determines how many bits are accessed simultaneously. Depending on the size, the bus transfers up to 16 specific bits in each access. The word lines are connected to the *row decoder* which is a multiplexer used to drive the word lines selected by the row address. Figure 2.2 illustrates how components in a memory bank are interconnected to form a memory array. Note that the number of rows and columns in this illustration is chosen arbitrarily and will vary for different DRAM chips.

2.1.1 Accessing the memory

When performing a read or write operation, the memory controller uses the physical address to determine the channel, rank, bank group, bank, row, and column that needs

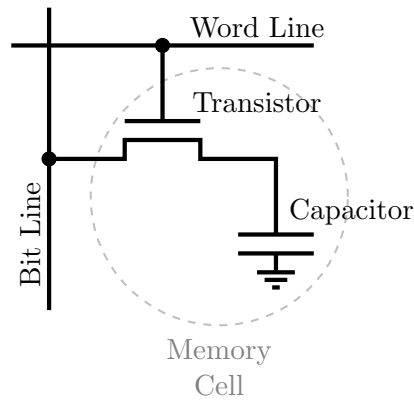


Figure 2.1: The structure of a single DRAM cell. The capacitor stores the charge of the cell. Through the transistor, the capacitor connects to the bit line. The gate of the transistor is connected to the word line.

to be accessed. It then sends a sequence of command signals to the memory module to execute the requested access.

To prepare for accessing memory, the selected bank is pre-charged. When charging a bank, a circuit raises the voltage on all bit lines to a potential halfway between logical '0' and logical '1'. Upon reaching the desired voltage, the pre-charging circuit disconnects, and the bit lines enter a *floating* state. Floating bit lines are not actively connected to a specific potential, but they still hold the previously applied charge for a short time due to their capacity. Setting the row and column address accesses the memory cells in a pre-charged bank. The row address and the column address use the same lines on the address bus to save space. Thus, the row and column addresses cannot be set simultaneously. Sending the *row-address strobe* (RAS) signal while asserting the row address to the address lines stores the row address in the *row address buffer*. The row decoder then drives the word line corresponding to the selected row address to a high signal. Applying voltage to the transistors of the memory cells in this row causes them to conduct current. This is called *activating* or *opening* a row [CJ10]. During activation, the cells of a row are connected to their corresponding bit lines. As a charge can flow freely through an open transistor, the charges of the floating bit lines influence the charges stored in the memory cells and vice-versa. A memory cell set to logical '0' drains charge from the connected bit line, while a cell set to '1' increases the voltage on the bit line by giving off its charge. Connecting a cell to a bit line is a destructive process, and the cell's charge has to be restored after every access. After a short charging time, the sense amplifier compares the voltage of the connected bit line to a reference voltage and amplifies the result to logical '0' or logical '1'. Through the amplification of the bit line charge, the memory cells are recharged by the sense amplifiers as a side effect, thus restoring the previously destroyed state of the cell. This recharging does not

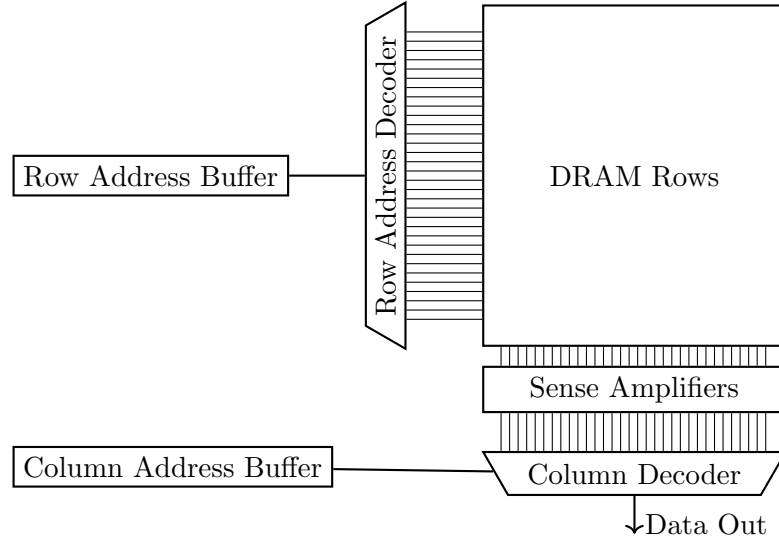


Figure 2.2: The schematic structure of a DRAM bank, reproduced from [JWN10]. The row address buffer and the column address buffer determine the accessed data. Through the decoders, the correct data words are wired to the output.

happen immediately. The row must stay active for a minimum time to ensure the correct restoration of all cell charges [JED13; JED17; JED20].

Following the row activation, the control circuit issues the *column-address strobe* (CAS) while asserting the column address on the address bus. With this signal, the column address buffer stores the asserted column address. Depending on the state of the *write enable* (WE) signal during the CAS, the operation is a write or read access. The access type may not change until the finalization of the memory transaction. By demultiplexing the column address, the column decoder connects a specific subset of the sense amplifiers to the data out of the chip. The number of connected sense amplifiers depends on the data width of the chip, and it ranges from 4 to 16 bits [JED13; JED17; JED20].

On write access, the connected sense amplifiers are driven to the logical values of the data applied on the module’s data bus. As the sense amplifiers charge the memory cells as long as the row is activated, forcing them to a specific value will also set the charge in the affected memory cells.

Figure 2.3 shows a simplified timing diagram for a single read operation [Sam14]. Grey areas indicate unknown signal levels. They are periods in which the value of the corresponding signal does not influence the operation. The voltage levels V_{high} and V_{low} represent logical ‘1’ and logical ‘0’, respectively. Note that the RAS, CAS, and WE signals are inverted. Inverted signals are set by setting the corresponding command bus line to low potential.

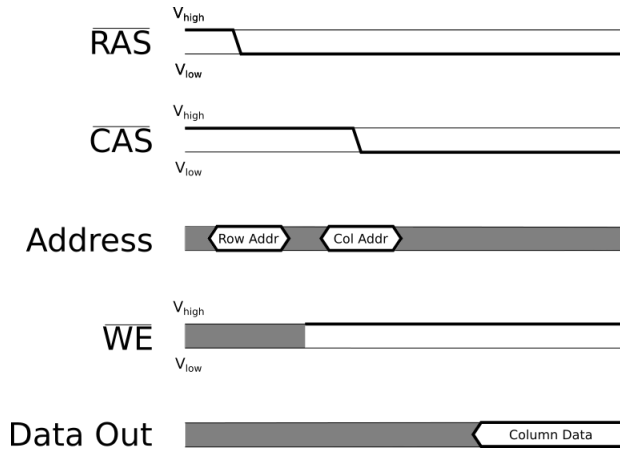


Figure 2.3: The simplified timing diagram of a DRAM read operation.

Once a row has been accessed, it stays open until the next pre-charge of the bank. With an active row in a bank, subsequent access to the same bank can lead to 2 possible scenarios:

Row hit Accessing a previously activated row is called a *row hit*. As the sense amplifiers already hold the row data, this type of access bypasses the pre-charging and the RAS phase. By sending the CAS signal while asserting a new column address, the column decoder connects a new subset of sense amplifiers to the data bus of the module. This change of connections takes only a short amount of time. Row hits are faster than other accesses to DRAM [Pes+16].

Row conflict The access targets a row that is not currently active. We call this scenario a *row conflict*. Before accessing a new row, the currently activated row has to be closed. Closing the row is equivalent to disconnecting the word lines and pre-charging the bank. If the currently active row cells are not completely recharged, the memory controller must postpone the closing operation to avoid data loss. Stalling the pre-charge leads to even longer latency during row conflicts. This type of access takes significantly longer than a row hit [Pes+16].

The memory controller can decide if a row is kept open or closed after each access. It thus enforces a so-called *row policy* [Awa+11]. We distinguish 3 possible policies:

Open row policy Rows are kept open until another row in the same bank is accessed. Doing so has the advantage that subsequent operations to the same row will always result in a row hit. Accessing a different row is slower, as the currently opened row needs to be closed beforehand.

Closed row policy After each access, the memory controller issues a pre-charge command, thus closing the row. When using this policy, accessing alternating rows in the same bank is sped up, as the bank will already be charged on the subsequent access.

Adaptive policy Depending on recent accesses, the memory controller decides whether to keep the row open or to close it. This policy aims to improve the performance by learning access patterns and optimizing row activation accordingly.

Modern memory controllers perform read operations in so-called *bursts* [JED13; JED17; JED20]. Instead of reading just one set of cells per memory access, a burst accesses multiple sets of cells in the same row in fast succession. The *burst length* determines how many separate reads are performed per burst. During a burst, the first access opens the target row. This row stays open during the access, even if a closed row policy is in use. All subsequent reads in the burst only need to modify the column address to access the relevant cells. Due to data locality, consecutive requests likely access locations close to each other in physical memory. Burst accesses exploit this principle by efficiently fetching more data than initially requested. If not all of the fetched data is needed, it can be discarded later on. In commodity systems, a burst fetches 64B. This is precisely the size of a cache line [Int21a]. Thus, burst accesses operate on cache-line granularity.

2.1.2 Refreshing the memory

As capacitors leak charge over time, charged memory cells in inactive rows will gradually lose their charge. If a memory cell discharges for long enough, the next read access to such a cell can produce incorrect results as the sense amplifier may not detect the correct logic level. To avoid data corruption due to leakage, the memory controller has to issue *refresh* commands periodically. The DRAM chip has to read, amplify, and write back the current charge to a cell during a refresh. These operations are the same as during row activation, so refreshing is always performed with row granularity. Note that for a refresh, it is only necessary to activate the row. Hence, sending a RAS and omitting the CAS is sufficient, as no data will be read or written. The leakage speed varies from cell to cell and strongly depends on the environment's temperature in which the DRAM operates. For temperatures ranging from 0°C to 85°C, JEDEC specifies the average *refresh interval* (tREFI) to be 7.8µs [JED17]. This leads to an average *refresh time* of 64ms for all rows of a DRAM module. The *retention time* is the time that a cell can hold its charge. Thus, it dictates the maximum period between two refresh operations on the same row. If the temperature exceeds the limit of 85°C, the retention time is halved.

Refreshing all rows simultaneously every 64ms would stall all other memory operations and would thus reduce the throughput. Instead, the memory controller distributes the refresh commands over time such that each row refresh satisfies the required retention time. Note that accessing a row will always cause a refresh. Thus, recently accessed rows will be refreshed later than rows that have been inactive for some time.

2.2 DRAM Faults, Rowhammer Mitigations and Integrity Protection

When using memory modules, we implicitly assume that the data retrieved during a read operation is the same data that a write operation stored previously. If this assumption is violated, we speak of a *DRAM fault* or *memory fault*. In general, we distinguish between two types of faults:

Transient faults Transient faults are soft errors in DRAM modules. Reading from a faulted cell will return an incorrect value; writing to the faulty cell fixes this type of error. This kind of fault is the predominant error type in DRAM modules.

Permanent faults This type of error persists over read and write accesses. A cell subjected to a hard error is stuck at '1' or '0' and will always return this value. Recovery from hard faults is not possible except by replacing the faulty component. Systems that can endure hard faults often employ redundant hardware and switch the faulty component during operation.

Theoretically, the memory cells of the DRAM modules should maintain data integrity indefinitely. This assumption only holds for perfect electrical components and optimal environmental conditions. In practice, electrical components are subject to fluctuations in production quality, deterioration, and adverse environmental effects. These influences cause the capacitors to lose their charge and, thus, potentially violate memory integrity. Two types of leakage mainly influence the charges stored in memory cells.

Gate-Induced Drain Leakage (GIDL) describes the effect in which electrons disperse from the capacitor towards the closed transistor [Cha+87; Oh+11]. This drainage means that a small amount of current flows towards the bit line even when the word line is not active. The amount of leakage does not depend on the capacitor but the materials and the geometry of the transistor. Due to electrons trapped in the silicon of the transistor, the GIDL may vary greatly during operation [Yan+87]. Such fluctuations lead to cells with *variable retention time* (VRT) [RPL92]. Even under constant environmental conditions, a VRT cell may spontaneously change its retention time.

The second leading source of electron leakage is the storage capacitor itself. Capacitors consist of two plates or surfaces of conductive material separated by a dielectric material. A perfectly dielectric material does not permit charge to flow freely from one surface to the other. Charging a capacitor creates an electric field in the insulator as the electrons aggregate on one side of the dielectric material. As the capacitors in DRAM cells are not perfect components, the dielectric material will conduct a small amount of charge, gradually draining the capacitor. The leakage current through the insulator strongly depends on the temperature of the capacitor [Ger+05]. Increasing temperatures will cause an exponential increase in the observed leakage. Thus, the JEDEC standard specifies that the refresh interval is halved above 85 °C. In long-running DRAM modules, repeated access to a memory cell causes degradation of the cell. This effect can cause hard faults in cells that operate for too long [Cho+06; Kim+01; Ban+88].

The core principle of data integrity in DRAM modules is that a cell needs to be refreshed before too much charge has leaked from it. Due to minor differences in the memory cells, each cell may have a different leakage speed. The refresh interval must be chosen such that even the cell with the shortest retention time is refreshed before losing too much charge. Selecting the refresh time according to profiled retention times is problematic, as VRT cells may spontaneously alter their behavior [RPL92]. Instead, the JEDEC standard specifies minimum retention times for specific temperature ranges [JED13; JED17; JED20]. DRAM manufacturers need to ensure that their modules conform to the standardized timings.

Even if all necessary timing constraints are fulfilled, it is still possible for natural errors to manifest in memory modules. The described aging effect may cause permanent faults in cells, especially in continuously operating DRAM modules [Huh+95]. In 1996, researchers found that cosmic radiation is the lead cause for naturally occurring transient faults in computer memory [Nor96]. High-energy neutrons colliding with the silicon in the memory cells can cause the stored bits to flip from '0' to '1' or vice versa. Such collisions are called *single event upsets* (SEUs). The change of stored data is called a *bitflip*. A neutron hitting the memory array may cause single-bit and even multi-bit errors. In the worst-case scenario, an SEU happens in the control logic of the DRAM chip. In such a case, the overall amount of bit errors increases drastically [BSS07]. The errors caused by SEUs greatly depend on the memory density. As memory density increases steadily, so does the overall amount of bitflips caused by radiation.

2.2.1 The Rowhammer Effect

Apart from naturally occurring bitflips, it is possible to cause bitflips in memory modules deliberately. In 2014, Kim et al. [Kim+14] reported that they were able to induce bitflips in DDR3 modules from unprivileged software. Their experiments found that repeatedly accessing alternating rows in the same bank, flips bits in physically adjacent rows. This effect was dubbed *Rowhammer*. Repeatedly closing and opening a row is thus called *hammering*. Rowhammer breaks the fundamental assumption that memory content only changes on write accesses, as it is possible to trigger the effect by just reading from memory. Furthermore, Rowhammer allows a malicious process to alter the memory content of other processes, thus bypassing memory isolation.

From a physical point of view, crosstalk between the aggressor and victim word lines is one of the major causes of the Rowhammer effect. This crosstalk raises the voltage of the victim word line, thus increasing the leakage through the transistor [WLB21]. When performing an attack, the attacker aims to cause as many row activations as possible to maximize the influence on the victim word line. Causing row conflicts in quick succession creates enough disturbance to induce flips into adjacent rows. Listing 2.1 shows the original code snippet proposed by Kim et al. [Kim+14]; this code snippet is enough to trigger the Rowhammer effect on a vulnerable DRAM module.

Two addresses, X and Y, need to be chosen to index different rows in the same bank. The rows indexed by these two addresses are called *aggressor rows*, and they are

```

1  loop:
2  mov (X), %eax
3  mov (Y), %ebx
4  clflush (X)
5  clflush (Y)
6  mfence
7  jmp loop

```

Listing 2.1: This snippet triggers the Rowhammer effect by accessing and evicting memory in rapid succession.

used to trigger bitflips in a *victim row*. Not all memory cells are equally vulnerable to Rowhammer, so it is difficult to predict potential victim rows from software.

In modern systems, the cache serves subsequent memory accesses to the same address. Hence, it is necessary to evict the aggressor rows between subsequent accesses. To avoid cache hits, cache lines can be evicted by using the `clflush` instruction. The `clflush` instruction evicts cache lines on command. Evicting the aggressor rows between accesses forces the memory controller to serve them from DRAM. In some systems, the `clflush` instruction might not be available or blocklisted. Hence, more sophisticated attacks use *eviction sets* [GMM16; AAA17]. An eviction set is a special sequence of memory addresses that, when accessed, evicts a specific cache line from the cache.

The `mfence` instruction serialized all previously issued memory accesses. Serialization is necessary to ensure that the accesses are performed before the next loop iteration.

To target specific rows in memory, we need the function that maps physical memory to rows. In general, this function is unknown, but it can be inferred using reverse engineering [Pes+16; Jun+16]. Note that this function depends on the CPU and the memory configuration of the system under attack.

Depending on the number of aggressor rows and their location relative to the victim row, different hammering patterns can be considered:

Double-sided hammering This hammering pattern aims to access two aggressor rows directly adjacent to the victim row. The reverse-engineered DRAM mapping function is needed to select the aggressor addresses accurately. When using double-sided hammering, the success rate is higher than with other hammering patterns. In general, this hammering pattern yields the best results regarding the number of induced bitflips [Gru+18].

Many-sided hammering The double-sided pattern can be extended by hammering additional rows in the same bank. When doing so, multiple victim rows are affected by the attack. Like the double-sided pattern, a many-sided pattern needs the DRAM mapping function. Additionally, one has to evict even more rows from cache than in the other patterns. Such a pattern might become infeasible for scenarios in which no `clflush` is available.

Single-sided hammering When using this access pattern, the attacker does not need to know the mapping function. This leads to the advantage that no exact knowledge of the system under attack is needed. Contrary to the name of the pattern, single-sided hammering does not access a single memory location. Instead, eight randomly selected addresses act as the aggressor rows. This pattern performs slightly worse than double-sided hammering [Gru+18].

One-location hammering In this hammering pattern, only a single memory location is repeatedly accessed. Initially, Kim et al. [Kim+14] observed no bitflips when accessing only one row in a bank. Repeated access to open rows will always result in a row hit on a system with an open-row policy. Thus, no crosstalk between the aggressor and the victim row should occur. Gruss et al. [Gru+18] found that if a closed row policy is in use, bitflips can be triggered by just accessing a single location. As the memory controller closes the row after each access, repeated accesses will behave similarly to row conflicts. One-location hammering is the most straightforward pattern, as no information on the system under attack needs to be known. Due to the dependency on the row policy, some systems might not be affected when using this pattern. One-location hammering performs worse than double-sided and single-sided hammering.

Self-evicting pattern De Ridder et al. [Rid+21] proposed this pattern to circumvent the use of the `clflush` instruction. It is a special case of the many-sided pattern. By allocating large memory regions, it is possible to create access sequences in which the aggressor rows act as their own eviction set. As addresses need to evict each other, precise knowledge on the function that maps physical addresses to cache locations is needed. Similar to the DRAM mapping, this function has to be reverse-engineered. When using a self-evicting set, each access hammers the memory while simultaneously evicting one of the other aggressor rows. Thus, the accesses to aggressor rows are always served directly from memory.

Figure 2.4 illustrates the three basic Rowhammer access patterns. Note that victim rows in the given figure are always directly adjacent to aggressor rows. In reality, a row activation influences all rows in the vicinity of the activated row. As the capacitive crosstalk decreases drastically with increasing distance, the directly adjacent rows are influenced the most and, therefore, are most susceptible for bitflips.

The bitflips induced by Rowhammer are random but reproducible. This means that it is hard to predict the exact memory cells that will flip before observing a flip. Once a cell is known to be vulnerable, it is possible to flip this bit reliably. The impact of a single bitflip on a system can be devastating, as demonstrated by various exploits. Seaborn et al. [SD15] exploited the bitflips induced by Rowhammer to gain kernel privileges from an unprivileged user. By filling the memory with page tables, they use the bitflips to modify the physical page number in the page table, thus accessing arbitrary memory.

Van Der Veen et al. [Vee+16] demonstrated an exploit that uses Rowhammer to gain root privileges on a smartphone. This attack shows that mobile devices with ARM chips are affected as well. It also shows that LPDDR3 DRAM is susceptible to Rowhammer.

2.2 DRAM Faults, Rowhammer Mitigations and Integrity Protection

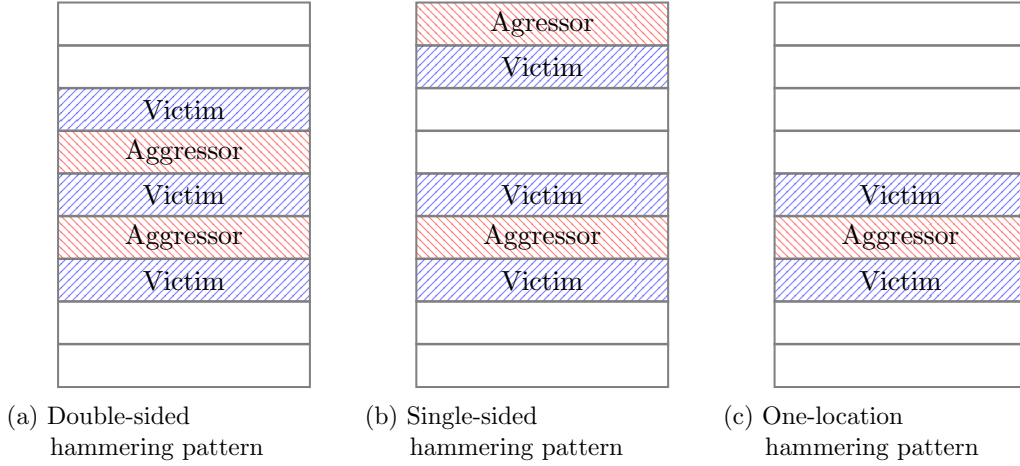


Figure 2.4: Illustration of different simple Rowhammer access patterns. Aggressor rows trigger bitflips in one or more victim rows. We distinguish the patterns by the relative placement of victim and aggressor rows.

More recent work showed that Rowhammer attacks are not bound to direct access to the target but can also be performed via malicious websites or over networks [GMM16; Lip+20; Tat+18b; Rid+21]. These attacks usually are more complex than local attacks, and they break the assumption that the attacker needs direct access to the system.

The success of a Rowhammer attack strongly depends on the system and the attacker’s capabilities. An attacker with full knowledge of the system might use double-sided hammering to achieve the maximum amount of bitflips. When the details of the system are unknown, one can only employ single-sided or one-location hammering. It is difficult to predict if a DRAM chip is susceptible and, if so, how many bitflips will occur. The results gained by Kim et al. [Kim+14] suggest that 80 % of DRAM modules are affected by Rowhammer. Thus, Rowhammer poses a significant threat to system security and data integrity.

2.2.2 Rowhammer Mitigations

As Rowhammer is a hardware bug, applying a patch or installing an update is not an option. Experiments by Kim et al. [Kim+14] suggest that DRAM modules ranging back to 2010 are vulnerable, indicating that a vast number of systems are affected by the bug. More recent studies indicate that, as DRAM chips become even denser, the number of vulnerable cells also increases [Kim+20]. Multiple defenses aim to reduce the security impact that Rowhammer has on modern systems. Gruss et al. [Gru+18] coarsely classify the proposed defenses into three types. According to the underlying approach of the defense, we differ between *detection-based*, *neutralization-based*, and *elimination-based* mitigations.

Detection-based mitigations When hammering a row, bitflips do not manifest immediately. Depending on the hammering pattern, it takes a varying amount of time until the first bitflips occur. Detection-based defenses aim to stop the attacking process before memory is compromised. For that, it is necessary to reliably identify and halt the process that is performing the attack. The fundamental assumption of all mitigations in this class is that Rowhammer attacks can be identified by monitoring specific metrics. Depending on the defense, these metrics differ.

Irazoqui et al. [IES18] propose MASCAT, a tool using static code analysis to detect potential microarchitectural attacks, including Rowhammer. By scanning the binary for `clflush` or `mfence` instructions, MASCAT can detect simple Rowhammer attacks and flag the malicious binary. Certain classes of eviction sets are also detectable. Thus, MASCAT may even detect attacks that do not use the `clflush` instruction. In theory, MASCAT can avoid the execution of malicious code altogether, thus preventing bitflips.

A popular way of detecting possibly suspicious activity is the use of *performance counters* [Awe+16; HF15]. These counters provide access to low-level CPU metrics, including the number of cache hits and misses per second. Multiple proposed defenses use the performance data to decide whether a process is malicious or benign. As Rowhammer attacks require uncached memory access [Kim+14], an attack will lead to an abnormally high number of cache misses. Through continuous monitoring, it is possible to detect such characteristic behavior and act accordingly. Benign programs may also cause a high cache load, so this metric should be combined with another metric to decrease the chance of false positives.

Apart from producing large numbers of cache misses, Rowhammer attacks all show unique access patterns [MK20]. By monitoring these patterns, it is possible to detect attacker code. Similar to the cache miss metric, memory access patterns should also be combined with another metric. Deciding if a program is malicious is a complex task, so one can expect that defenses based on performance counters tend to misclassify some processes. Such misclassification can flag benign programs as malicious or, even worse, miss a Rowhammer attack.

Neutralization-based mitigations Contrary to the previous defense class, mitigations that are neutralization-based aim to minimize the impact of the induced bitflips. In a Rowhammer attack, it is important for the attacker to find bitflips at *exploitable* locations. Randomly finding exploitable flips is rather unlikely. Thus, the attacker needs to place vulnerable memory on exploitable locations. Usually, this is done with a technique called *memory massaging* [Raz+16]. It is possible to force the operating system to hand out memory in locations with known exploitable bits through elaborate memory allocation. Privilege escalation attacks like the one presented by Seaborn and Dullien [SD15] aim to induce faults in kernel memory. Such attacks are only possible if userspace rows are physically adjacent to kernel memory rows. Isolating the address spaces in memory does not stop bitflips from appearing but reduces the possible impact.

Brasser et al. [Bra+16] implement this isolation in their defense *G-CATT*. By enforcing physical distance between kernel memory and user memory, they drastically reduce the

2.2 DRAM Faults, Rowhammer Mitigations and Integrity Protection

attack surface of Rowhammer. G-CATT divides the physical memory into so-called *security domains*. Different security domains are physically separated, meaning that an attacker can only induce flips in rows that belong to the same domain as the aggressor rows. The domains are isolated by a single row that will never be mapped to any domain. Thus, the memory overhead is constant and only depends on the number of rows per bank.

Konoth et al. [Kon+18] propose ZebRAM as a possible Rowhammer mitigation. This software-based protection divides the DRAM into secure and insecure rows. The division is done in an alternating pattern such that every even row is secure and every odd row is insecure. ZebRAM does not prevent bitflips in insecure regions. Instead, it performs integrity checks to detect and correct errors in such rows. ZebRAM is based on the assumption that Rowhammer only affects directly adjacent rows. Zhang et al. [Zha+19] show that this assumption is incorrect, as they induce bitflips up to a distance of two rows. Thus, they circumvent ZebRAM and induce bitflips in allegedly protected rows.

Elimination-based mitigations Protection schemes in this category aim to mitigate Rowhammer at its roots by preventing bitflips from manifesting. The proposed approaches range from software-only mitigations to modifications of the refresh logic of the memory controller. As of now, this is the largest class of mitigations.

One of the most basic approaches is to double the refresh rate of DRAM chips. Reducing the time between row refreshes significantly reduces the bitflip probability. Doubling the refresh rate comes with the downside that energy consumption rises while DRAM performance decreases. Unfortunately, experiments show that even with a doubled refresh rate, Rowhammer attacks can induce bitflips [AAA17; Awe+16] and that this solution does not scale well for newer DRAM generations [Kim+20].

Aweke et al. [Awe+16] propose ANVIL as a possible software-only defense. ANVIL is split into a *detection* and a *protection* phase. Similar to mitigations in the detection-based mitigation class, this defense uses a combination of performance counters and memory access patterns to decide if a row is under attack during the detection phase. Initially, ANVIL only monitors the cache miss rate. Once the number of cache misses exceeds a certain limit, ANVIL starts to track memory accesses to identify possible aggressor rows. If the sampled accesses show high locality, the protection phase is invoked. In the protection phase, ANVIL issues read requests to potential victim rows. These read requests essentially refresh the victims and prevent their cells from losing too much charge.

This approach fails on machines that do not offer the performance counters that ANVIL requires. Furthermore, Tatar et al. [Tat+18a] show a practical attack that completely circumvents ANVIL.

In their 2014 paper, Kim et al. [Kim+14] introduce PARA, a probabilistic defense against Rowhammer. The core idea of PARA is that each row activation triggers the activation of adjacent rows with some small probability p . By modifying this probability, it is possible to fine-tune the expected level of protection and performance impact. While

PARA might reduce the probability of bitflips, the authors state that it cannot protect against Rowhammer induced bitflips with absolute certainty.

Additionally to G-CATT, Brasser et al. [Bra+16] propose B-CATT. This bootloader extension aims to defend against Rowhammer attacks by blocklisting vulnerable memory locations. During booting, B-CATT profiles the system memory. It essentially performs a Rowhammer attack and logs all susceptible rows. The operating system will then only use memory locations that did not experience bitflips in the profiling phase. This approach has significant drawbacks. The profiling phase is not guaranteed to identify all vulnerable cells, as VRT cells may spontaneously become vulnerable. Thus, it is also not guaranteed that the system only uses non-vulnerable memory rows. Additionally, Kim et al. [Kim+14] found that in some DRAM modules, all rows have at least one vulnerable cell. For such modules, blocklisting almost all of the available memory would have a devastating effect on system performance.

Yağlikçi et al. [Yağ+21] present BlockHammer, a countermeasure that throttles accesses before they can induce bitflips. Using counting bloom filters, they keep track of row activations on a per-bank level. Once the number of row activations exceeds a pre-defined threshold, the row is blocklisted and throttled. The memory controller serves subsequent accesses to a blocklisted row with a fixed time delay. Parameters like the row activation threshold and the delay between accesses are modifiable, thus allowing for easy adaption to different DDR standards. As this defense only requires modifications in the memory controller, it is usable with various commodity DRAM modules.

The JEDEC standard for LPDDR4 modules addresses Rowhammer by officially introducing *Target Row Refresh* (TRR) and *Maximum Activation Count* (MAC) [JED17]. Activating a row more than MAC times between two refresh cycles places the DRAM in TRR mode. This mode refreshes all rows adjacent to target rows that exceed their MAC. According to the standard, TRR only considers directly adjacent rows as potential victim rows. Thus it suffers from the same problems as the previously mentioned ZebRAM mitigation.

Many of the proposed Rowhammer mitigations were already proven to be ineffective. Gruss et al. [Gru+18]. showed that even a combination of mitigations from different classes is not enough to protect against sophisticated Rowhammer attacks. Detection-based mitigations can be circumvented by running the attacker code in a secure enclave. Defenses that separate kernel and user memory are insufficient if the attacker explicitly targets userspace code, and mitigations based on memory profiles can easily be tricked by specific hammering patterns.

Even the state-of-the-art mitigation TRR cannot sufficiently protect against Rowhammer. Frigo et al. [Fri+20] show that TRR implementations can only track a subset of all row activations. They exploit this limitation with module-dependent hammering patterns that trick the mitigation and introduce bitflips. Based on this work, de Ridder et al. [Rid+21] show that TRR can even be circumvented from JavaScript, proving that it is not an effective Rowhammer mitigation. Researchers at Google take a different approach and exploit the assumption that hammering a target row only influences adjacent rows. They introduce *half-double*, a novel hammering pattern that attacks non-adjacent victim

rows [Sal21]. With this pattern, they can induce bitflips despite TRR being active on the system under attack.

2.2.3 ECC and Hamming Codes

As discussed above, DRAM is susceptible to single and multi-bit errors. While our proposed mitigations may avoid some Rowhammer induced bitflips, they fail to protect against naturally occurring faults. Thus, systems that handle important data or are hard to maintain need some kind of error detection and correction mechanisms. For DRAM modules, these mechanisms are so-called *error-correcting codes* (ECCs) [Ham50]. Error correction and detection is based on redundancy. This means that in addition to regular data, redundant information computed from the data is stored. With this additional data, an error-correcting code can detect and possibly correct bit errors. Naturally, this leads to some additional storage requirements, the so-called *memory overhead*. In general, error-correcting codes aim to detect and correct as many errors as possible while keeping the memory overhead to a minimum.

The simplest way of detecting single-bit errors is by using a *parity bit*. Given a data word W with n bits b_i , the parity of the word can be computed as

$$p(W) = \bigoplus_{i=1}^n b_i \quad (2.1)$$

where $\oplus$ is a binary XOR-operation. The result of a parity computation can be *even* (0) or *odd* (1). Given a data word W and a pre-defined parity specified by $p_{\text{specified}}$, the parity bit b_p is computed such that

$$p(W) \oplus b_p = p_{\text{specified}} \quad (2.2)$$

holds. Essentially, the parity bit is a one-bit checksum. When accessing a word and its parity bit, Equation 2.2 computes the parity of the word and the check bit. In the case of a single-bit error, the computed result will not match the specified parity. Thus, the error is detected. According to Equation 2.1, the computed parity of a data word only changes if an odd number of bits in the word are changed. Hence, simple parity bits cannot detect errors that do not change the outcome of the parity computation. Even if an error is detected, a single check bit is not enough to perform correction, as there is no possibility to deduce the location of the error.

The number of detectable bit errors is directly tied to the *Hamming distance*. It describes the bitwise distance between two valid codewords [Ham50]. When using a simple parity check, a codeword is considered valid if the overall parity is equal to the specified parity. A single parity bit leads to a Hamming distance of 2, as a change of two bits will create a new, valid codeword. Given a code with Hamming distance d , up to $d - 1$ errors are detectable.

Hamming codes extend the simple parity checks to allow for error correction [Ham50]. Instead of having one parity bit for the whole codeword, a Hamming code needs multiple parity bits for overlapping sections of the codeword. The sections are chosen such that

Table 2.1: Memory overhead for different SEC and SEC-DED codes

SEC			SEC-DED		
n [bit]	k [bit]	O [%]	n [bit]	k [bit]	O [%]
4	2	100	5	2	150
7	4	75	8	4	100
12	8	50	13	8	62.5
21	16	31.25	22	16	37.5
38	32	18.75	39	32	21.88
71	64	10.94	72	64	12.5
136	128	6.25	137	128	7.03

each possible single-bit error will lead to a unique pattern in the parity bits. When using error-correcting codes, a codeword of size n is made up of k data bits and d parity bits such that $n = k + d$. During validation, the *syndrome* is computed from the d stored parity bits and the d parity bits computed on the current data. The result is again a d -bit value, indicating if and where an error has occurred. It follows that the syndrome can cover at most 2^d possible states. One of these states indicates that no error occurred, so a d -bit syndrome can indicate $2^d - 1$ erroneous states.

Bit errors can occur in the data and the parity bits. An error-correcting code has to account for that by ensuring that the syndrome can cover all possible cases. From that constraint, it follows that

$$k + d \leq 2^d - 1 \Leftrightarrow n \leq 2^{n-k} - 1 \quad (2.3)$$

must hold. Violating this constraint leads to codes that cannot correct all possible single-bit errors for the given number of data bits.

Extending a single-error correcting (SEC) code with an additional parity bit leads to a single-error correction, double-error detection (SEC-DED) code [Sta03]. The extra bit acts as a parity bit over all other data and parity bits. In the case of a double error, the correction mechanism of a regular SEC code would introduce one additional erroneous bit as it cannot distinguish between a double error and a single-bit error of some different bit. A SEC-DED code can detect such an error and report it to the system. According to the definition given above, a SEC code has a Hamming distance of 3, and a SECDED code has a distance of 4.

Error-correcting codes are usually denoted as (n, k) -Hamming codes. This (n, k) tuple specifies the codeword length and the number of data bits per codeword. With these two values, one can compute the percentual memory overhead O of a code as

$$O = 100 \cdot \frac{n - k}{k}. \quad (2.4)$$

With Equation 2.4 one can compute the overhead for optimal Hamming codes with different data sizes. Table 2.1 shows the results for different configurations. It is clear

2.2 DRAM Faults, Rowhammer Mitigations and Integrity Protection

to see that adding an additional parity bit doubles the number of possible data bits. This relation indicates that error-correcting codes perform especially well w.r.t. memory overhead when used with larger codewords.

Hamming codes have the advantage that they are relatively easy to implement in hardware. The circuit computing the syndrome can even be as simple as d XOR gates. Constructing the error detection and correction with logic gates dramatically increases the speed with which an actual ECC module operates.

ECC DRAM modules are widely available and commonly used in servers or computing farms. They do not comprise the logic circuitry needed to detect and correct bit errors. Instead, ECC DRAM offers additional storage for checksum bits. In commodity modules, manufacturers add one additional chip per rank such that the bus size increases to 72 bit [JED13; JED17]. With these additional 8 parity bits, a (72,64) Hamming code can be used to detect and correct errors. Each read access to an ECC DRAM module yields a 64-bit data chunk and the accompanying 8 checksum bits. From Table 2.1, we can see that this configuration gives a memory overhead of 12.5%. The (72,64) configuration is efficient as it allows manufacturers to use the same kind of chip for the parity bits already used for regular data. Modules that increase the bus width use *side-band ECC*, as all parity bits are transmitted simultaneously with the data bits. It is possible to use ECC in combination with non-ECC DRAM, this technique is called *in-band* or *in-line* ECC. By storing the parity bits as part of the data, no additional chip is needed. This has the disadvantage that one data access leads to multiple read accesses. Also, the usable DRAM capacity would only be 87.5% of the overall capacity. Using an in-line solution can be viable if increasing the bus width would not be possible.

As current ECC DRAM modules only provide a wider data bus and additional storage space for parity bits, combining such a DRAM with a compatible mainboard and CPU is necessary. Components that support ECC will implement the detection and correction logic as described above. They do not necessarily use the discussed Hamming code but some proprietary derivation of it.

According to the JEDEC standard and manufacturer data [RK19; JED20], DDR5 implements on-chip ECC. The proposed variant uses 136-bit codewords with 128 data bits and 8 parity bits. As can be seen in Table 2.1, this configuration reduces the memory overhead by half but only allows for single-error correction. Multi-bit errors still may cause incorrect corrections and are not guaranteed to be detectable.

While systems supporting ECC are hardened against single and possibly double bitflips, research shows that they are still vulnerable to Rowhammer attacks [Coj+19]. By flipping a number of bits that is not detectable by the ECC, it is possible to induce bitflips without triggering the detection and correction mechanisms. In general, the performance of ECC strongly correlates with the Hamming distance of the deployed code. As soon as the number of bitflips exceeds the margin given by the Hamming distance, bitflips stay undetected or, even worse, are corrected in the wrong direction.

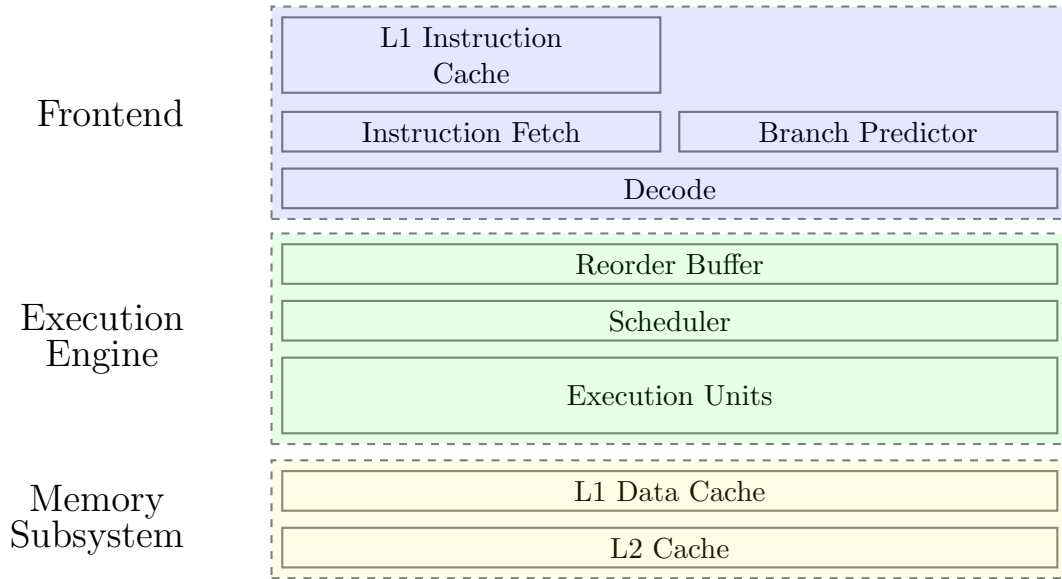


Figure 2.5: The schematic structure of a single CPU core. Each core is split into frontend, execution engine and memory subsystem.

2.3 Intel x86-64 CPU Architecture

This section briefly discusses the general structure of modern Intel processors, the memory controller, and the vector instruction set.

Intel’s two most prominent processor families are the Intel Core and the Intel Xeon family. Intel Core products are commonly used as desktop and mobile processors, while Xeon CPUs find use in server applications and workstations. Regardless of their field of use, the internal structure of all Intel CPUs is roughly the same.

Each CPU has one or several cores. These cores fetch, decode and execute the instructions of the operating system and the running processes. Internally, a core consists of multiple subsystems that operate in parallel. Figure 2.5 illustrates the simplified microarchitecture found in Intel CPUs.

Each core can be divided into the *frontend*, the *execution engine* and the *memory subsystem* [Int19a].

The main task of the frontend is fetching and decoding instructions. Modern CPUs do not wait until a fetched instruction has been executed to fetch the next instruction. Instead, code is *pre-fetched* and partially processed while the current instruction is still executing. In the case of branching code, the *branch predictor* decides which instructions to fetch. Recently loaded instructions are directly stored in the *L1 instruction cache*, as the principle of locality suggests that recently executed code might be executed again in the near future. Loaded instructions are inserted in the *instruction queue*. The size of this queue dictates the maximum amount of pre-fetched instructions. From this queue,

the decoder loads and deconstructs the instructions into *micro-ops* (μ -ops). These μ -ops are the instructions that the CPU can actually execute. The CPU hands them over to the execution engine.

In the execution engine, the *reorder buffer* (ROB) receives, stores, and analyses the μ -ops emitted by the frontend [Int19a]. According to the data dependencies of the operations, the ROB may change the order of execution. This reordering increases the throughput by allowing independent operations to execute in parallel. A *scheduler* distributes the instructions among all available *execution units*. As execution units are specialized circuits, the scheduler needs to allocate the units according to the issued instructions. Execution units that handle load and store operations directly connect to the memory subsystem. Due to branch prediction, instructions may be executed speculatively. Due to speculation, the execution engine might process instructions from a code branch that should not be executed at all. Resolving the branch condition indicates if the branch predictor speculated correctly. *Retiring* executed μ -ops from a correct branch stores the computed results in memory or the CPU registers. If the branch is mispredicted, all pre-fetched instructions are flushed. Flushing the instructions discards all speculatively executed μ -ops.

Execution units that handle memory operations interact with the memory subsystem. This subsystem comprises the L1 data cache and the L2 cache. Both caches aim to reduce access latencies by minimizing the number of accesses served from the main memory. L1 caches are always private, meaning that they are not shared between multiple cores.

Each core owns a so-called *slice* of the L3 cache. Contrary to L1 and L2, the L3 cache is shared over all cores [Int19a]. Depending on the CPU family, the cache slices are connected via a ring bus or a mesh grid interconnect. This interconnect allows a core to access data stored in cache slices of other cores. All CPU components that are not associated with a specific core are *uncore* components [Int21b]. Hence, the bus serves as the connection between the core and the uncore parts of a CPU.

2.3.1 The Memory Controller

One important uncore component is the memory controller. It manages all low-level interactions with the DRAM modules connected to the physical channels. As the cores do not directly communicate with the DRAM, the memory controller has to handle several tasks:

Address Translation Each access to a physical address needs to address the correct channel, rank, bank, and row. The mapping function linking physical addresses to DRAM rows is proprietary and generally unknown [Pes+16]. Research shows that the mapping depends on the type of CPU and the current memory configuration. For memory sizes that are a power of two, linear functions map the addresses to their physical locations. Other memory configurations that result in non-power-of-two sizes result in non-linear functions. Reversing such functions takes considerably more effort than in the linear case, and until now, no completely generic method is available.

As discussed in Section 2.2, the mapping function is an essential part of sophisticated Rowhammer attacks. Without this function, hammering patterns such as double-sided hammering or many-sided hammering are practically infeasible.

Access Scheduling Similar to the reorder buffer in the core, the memory controller can change the order of issued memory accesses. As DRAM modules in different channels can operate in parallel, the memory controller schedules subsequent accesses such that they are spread as much as possible to increase the overall throughput [Rix+00]. For a single module, the controller may change the access order to avoid row conflicts to decrease the latency. Additionally, the controller may group multiple accesses that target the same row to a burst. As memory controllers are proprietary components, the access scheduling algorithms are undocumented.

Since the scheduling may change the order of row activations, the reordering of memory accesses also influences Rowhammer attacks. A reordering that aims to increase subsequent row hits might complicate attacks as the number of row activations will decrease.

Memory Access Accessing data in DRAM requires several steps, as described in Section 2.1. The memory controller emits a sequence of control signals that cause the desired memory operation in the DRAM module. After completing a read request, the interconnect transports the fetched data to the requesting core.

DRAM Refresh As described in Section 2.1, frequently refreshing DRAM rows is necessary to maintain data integrity. In general, DRAM modules do not perform these refreshes automatically. Instead, the memory controller has to issue periodical row activations. The refresh rate may change over time and can be fine-tuned [JED20]. With additional sensors, the memory controller can perform thermal throttling if the DRAM temperature increases too much. By raising the refresh rate, the increased leakage can be compensated. Increasing the memory density of DRAM modules leads to more rows that the memory controller needs to activate in each refresh cycle. Depending on the internal structure of the memory array, multiple rows in a bank can be activated in parallel. Different banks must be activated sequentially, as they share parts of their circuitry. The number of simultaneously activated rows is chosen such that the overall charging current does not exceed limitations. For DDR4 modules, *fine-grained refresh* (FRG) aims to balance the number of activated rows against the refresh overhead and the average DRAM stall time [JED20].

Integrity Checking In configurations that offer ECC support, the memory controller handles the computation and evaluation of checksums. As SEC-DED codes are easy to implement in hardware, the timing overhead induced by these integrity checks is minimal. Depending on the size of the additional circuitry, the power consumption may increase compared to non-ECC memory controllers. In the case of error-free operation and single-bit errors, memory accesses behave transparently to the operating system.

Prefixes	REX Prefix	Opcode	ModR/M	SIB	Displacement	Immediate
0 - 4 Byte	0 - 1 Byte	1 - 3 Byte	0 - 1 Byte	0 - 1 Byte	0 - 4 Byte	0 - 4 Byte

Figure 2.6: Intel 64 Legacy Instruction Format, reproduced from [Int21b]. Each instruction consists of up to 7 fields.

As double bit errors are uncorrectable with SEC-DED codes, they are treated as machine check errors. Such errors cause a machine check exception, a special type of interrupt that usually leads the operating system to shut down.

Memory Scrubbing Some ECC implementations use memory scrubbing to detect and correct single-bit faults before the faulty data block is accessed. Between regular accesses, the memory controller can sweep over the physical memory and perform dummy read accesses. The goal of these accesses is to detect and correct single-bit errors before another error appears in the same block. Scrubbing essentially reduces the probability of double bit errors arising from multiple single-bit errors in the same block.

2.3.2 Instruction Encoding, Vector Registers, and the AVX-512 Instruction Set

Modern Intel processors comply with the so-called *x86-64 Instruction Set Architecture* (ISA). This ISA specifies a common set of instructions that every x86-64 processor has to implement. Each instruction is defined by a unique *opcode* and additional fields.

Figure 2.6 illustrates the Intel 64 instruction format for 64-bit mode according to the Intel 64 and IA-32 Architectures Software Developer’s Manual [Int21b]. Due to the variable size of some of the fields, the Intel 64 architecture uses variable-length instructions. Varying the instruction size has the advantage that each instruction only takes as much space as it needs. As can be seen in the illustration, each instruction comprises a maximum of 7 fields when using the legacy instruction format. The Intel Instruction Set Reference [Int19b] describes the fields as follows:

Prefixes Prefixes are optional and change the behavior of the instruction. In commodity Intel processors, prefixes are divided into four groups. At most, one prefix of each group can be present simultaneously in arbitrary order. Notable prefixes are the *LOCK*, *branch hints*, the *operand-size override* and the *address-size override* prefix. *LOCK* allows for atomic execution of certain instructions. Branch hints indicate the most likely code path and thus help the branch predictor to avoid misprediction. Using the override prefixes alters the address and operand size from 16 to 32 bit. Note that some of the prefixes have a second use as a mandatory prefix for certain instructions.

REX Prefix If needed, this special prefix must be the last prefix before the actual opcode and, when applicable, follow the mandatory prefix. Some instructions require the REX byte in 64-bit mode if they do not default to 64-bit operands. The upper 4 bits of this prefix byte are always fixed to 0x4. The lower 4 bit can be modified, resulting in a range from 0x40 to 0x4f. REX can modify the operand size or modify the meaning of the following fields depending on the previous prefixes.

Opcode The opcode determines the actual instruction that is executed by the CPU. Each opcode can have one to three bytes, and opcodes of similar operations are grouped together. In multi-byte opcodes, certain values, so-called *escape opcodes*, change the meaning of the following bytes. Mandatory prefixes and escape codes help to increase the number of possible encodable operations.

ModR/M The meaning of the contents of this byte strongly depends on the preceding opcode. Internally, the ModR/M can be split up into the *mod* field, the *reg/opcode* field and the *R/M* field. Depending on the combination of the fields, ModR/M can specify additional opcode information, the addressing mode, or register operands.

SIB SIB stands for *scale - index - base*. Some configurations of ModR/M require the SIB byte for a complete encoding of the addressing mode.

Displacement These bytes define an offset on the encoded operand address. The addressing specified by the ModR/M in combination with the SIB decides if a displacement is needed.

Immediate Immediate values are encoded in the last one to four bytes of the instruction. Depending on the type of operation, this field can be left out, as not all instructions require an immediate value.

While this instruction encoding covers all general-purpose operations, Intel introduced multiple *instruction set extensions* [Int12]. These extensions add new specialized instructions and instruction encodings to the existing schemes.

In older processors with Intel's SSE, the vector register size was limited to 128 bit. With the introduction of *Advanced Vector Extensions* (AVX), Intel increased the size of these registers to 256 bit. Additionally, AVX adds new instructions and a new instruction encoding scheme to the legacy instruction set. With AVX, the REX prefix becomes obsolete and is replaced by the VEX prefix. Contrary to REX, the VEX prefix is two or three bytes in size. Hence, opcodes may only take up one byte with this encoding. This limitation does not reduce the number of possible opcodes as single bits in the VEX prefix represent the escape opcodes. Using the encoding with VEX is optional for legacy instructions but mandatory for instructions that operate on 256-bit vector registers.

Similar to AVX, AVX-512 extends the pre-existing registers and instruction set. With the introduction of this extension, vector register size is increased to 512 bit, and a new prefix is added. The *EVEX* prefix allows for a third type of instruction encoding that

is fully compatible with all legacy instructions. Contrary to the VEX prefix, EVEX is always four bytes in size. With the new encoding, it is possible to encode conditional operations directly in the instruction.

The main purpose of vector registers are *single-instruction-multiple-data* (SIMD) operations. In such instructions, the same mathematical operation is applied to all entries of the vector register in parallel. This type of parallelism notably increases the performance during digital image or audio processing.

As Intel processors are backward compatible, additional vector registers comprise existing registers. The initial set of SSE registers, the 128-bit XMM registers, form the lower half of the 256-bit YMM registers introduced with AVX. Likewise, each 512-bit ZMM register is built upon a YMM register. This scheme allows for legacy code to run on a newer processor without changing the needed instructions.

2.4 Interrupt and Exception Handling in the Linux Kernel

During regular code execution, *interrupts* and *exceptions* may alter the execution flow spontaneously [BC05]. While both types of events behave similarly, there are some key differences between handling interrupts and exceptions. As hardware signals may trigger interrupts through *interrupt requests* (IRQs), they can occur asynchronously at any time during regular operation. Contrary to that, exceptions are only triggered synchronously when an error is detected while executing an instruction.

In modern Intel processors, each core has a local *advanced programmable interrupt controller* (APIC) [Int21b]. This controller directly handles interrupts from core-specific hardware and I/O devices, thermal sensors, performance counters, and APIC-internal sources. An additional I/O-APIC that is shared among the available cores handles I/O-related interrupts and inter-processor interrupts. One of the main tasks of the shared APIC is to distribute I/O interrupts to the local APICs such that each core handles approximately the same interrupt load. The interrupt messages generated by the I/O APIC are distributed to the specific cores via the system bus.

As the CPU directly generates exceptions, no additional hardware is needed. Naturally, an exception only concerns the core in which it is detected.

When receiving an IRQ or an exception, the processor transfers execution to the corresponding interrupt handler or exception handler. This handler then reacts appropriately to the interrupt or the exception. Once an interrupt handler finishes executing, the previously running process resumes at the instruction where it was preempted. For exception handlers, this is only true if recovery from the exception is possible. Programming errors like division by zero or invalid memory accesses may lead to program termination.

Depending on the way they are handled, exceptions can further be subdivided into *faults*, *traps* and *aborts* according to the Intel Developer Manual [Int21b]. A fault is a type of exception that the kernel can potentially correct. Faults may occur if a user program accesses unmapped memory (page fault) or if privileged memory is accessed by an unprivileged process (segmentation fault). Depending on the circumstances and the fault type, normal program execution may continue after the fault. If a fault is

resolved and execution is resumed, the next instruction executed after a fault is the same instruction that caused the fault in the first place.

Traps are particular instructions that deliberately cause exceptions in the CPU. They can be used to implement breakpoints in debuggers or to perform syscalls. Once the handler of a trap finishes executing, code execution resumes at the next instruction after the trap instruction.

Aborts indicate an unrecoverable error. On such a signal, the abort handler potentially performs cleanup operations and halts the executing process. As it is impossible to recover from an abort, this exception type does not reliably store the program counter like the other exception types.

External interrupts that are issued by I/O devices can be grouped into *maskable* and *non-maskable* interrupts (NMIs). Maskable interrupts can be disabled, meaning that the CPU will ignore a masked interrupt. On the contrary, non-maskable interrupts can never be ignored and need to be served immediately. Such interrupts usually indicate critical hardware failure and need to be handled accordingly. While external interrupts may occur asynchronously, they are always guaranteed to be served at an instruction boundary. This synchronization allows the CPU to save the instruction pointer of the preempted task for later continuation.

In x86 systems, each handler needs to be registered in the so-called *interrupt descriptor table* (IDT) [BC05]. This table resides in kernel memory and is populated during booting and system initialization. Each CPU core has its exclusive IDTR register that points to the address of the corresponding IDT. Thus, theoretically, each core can have its own IDT. 64-bit Linux uses one IDT with 256 16-byte entries. Each entry in the IDT is called a *gate* and holds information on the corresponding interrupt or exception. The index of each IDT entry is called *vector* and vector values range from 0 to 255. When an interrupt occurs, the local APIC has to translate the IRQ to a vector number using its *local vector table* (LVT). This vector is then subsequently used to index the correct gate. Vector numbers for exceptions are hard-wired and do not require an additional translation.

Depending on the type of handler, each gate can either be a trap-gate or an interrupt-gate. When entering a handler function through an interrupt gate, local maskable interrupts are automatically disabled. For handlers that do not need to run atomically, it is thus necessary to explicitly re-enable interrupts in the handler function. As specified in `desc_defs.h`, a single IDT entry contains multiple fields that determine the handler's address and additional information. Figure 2.7 shows the structure of a gate descriptor.

In 64 bit systems, the upper 32 bit of each gate descriptor are reserved for future use. The *offset* fields in the descriptor determine the kernel address at which the corresponding handler is located. By combining all three offset fields, one can construct the absolute address of the handler function assigned to the specific gate. The upper 8 bits of the *IDT bits* determine if the handler is in use, the gate type, and the privilege level needed for this gate. In current systems, the lower 8 bits remain unused and are reserved for future purposes. Finally, the 16-bit *segment* field points to an entry in the *global descriptor table* which in turn holds information on the location of the memory segment of the handler.

2.4 Interrupt and Exception Handling in the Linux Kernel

Reserved	Offset High	Offset Middle	IDT Bits	Segment	Offset Low
32 Bit	32 Bit	16 Bit	16 Bit	16 Bit	16 Bit

Figure 2.7: Gate descriptor data according to the Linux kernel sources. Each IDT entry is a gate descriptor. They determine the how interrupts are handled.

Table 2.2: Content of the per-CPU IDT according to the Linux kernel sources

Vector	Description	Type
0 - 21	Exceptions and NMIs	Traps, Faults, Aborts and NMIs
22 - 31	Intel Reserved Interrupts	Traps
32 - 127	Device Interrupts	Maskable Interrupts
128	Syscall Interface	Maskable Interrupt
32 - 255	Device Interrupts	Maskable Interrupts

In the Linux kernel, the files `irq_vectors.h` and `trapnr.h` define the vectors as can be seen in Table 2.2. The vectors correspond directly to the priority of the handler. A low vector number indicates a high priority, while high vector numbers numerate low-priority external interrupts. These priorities are important when multiple interrupts and exceptions are pending. They determine which handler will be executed first.

When setting up the IDT entries, Linux uses the `idtentry`-macro found in `entry_64.s`. This assembly macro generates the same entry stub for all handlers registered in the IDT. As some interrupt and exception sources automatically push an error code on the stack, the macro accounts for that by pushing a dummy error code for events where that does not happen. By doing so, the code snippet guarantees that the stack frame has the same layout for each handler. Like a regular context switch, the macro also saves the current set of registers as they need to be restored once the handler finishes. By wrapping the call to the actual handler function with the `error_entry` and `error_return` function, the correct arguments are passed to the handler, and once it has finished, the registers are restored to allow execution to continue. If an exception handler detects an unrecoverable situation in a user process, it sends a signal indicating the exact type of error to the process.

During system operation, the first 32 entries of the IDT must be mapped and available. These vectors numerate the highest priority interrupts and include exception handlers for system-critical events like zero divisions, machine check errors, and page faults.

In the IDT, it is not necessary to discern between faults and traps in the definition of the handlers. For faults, the CPU stores the instruction pointer of the instruction that caused the fault. In the case of a trap, the stored instruction pointer points to the next instruction after the instruction that caused the trap.

2.4.1 Page Fault Handling

One of the most common faults occurring during regular system operation is the page fault. Page faults are triggered by accesses to unmapped memory areas, memory accesses with incorrect permission, when trying to execute code from a non-executable memory region, when writing to a read-only page or if certain reserved bits in the paging structure are set [BC05]. On processors that support SGX, it is also possible that page faults occur due to enclave access violations. Every time a page fault is triggered, the memory management unit of the CPU stores the accessed address that caused the page fault to happen in the core's *control register 2* (CR2) [Int21b]. Additionally, the CPU pushes a 32-bit error code on the stack. This error code holds information about the access type that caused the page fault. The number of used error bits depends on the features that the CPU supports. In general, each error code uses at least 5 and up to 8 of the 32 bits. Encoding information like the cause of the page fault, the access type, and the privilege level during the access allows the handler to differ between valid and invalid page faults.

As soon as the page fault handler is entered, the content of the CR2 is stored to a variable on the stack of the fault handler. This is necessary as each subsequent page fault on the same core overwrites the contents of the CR2. When entering the handler function, local interrupts are disabled as it is entered through an interrupt gate. Re-enabling local interrupts early in the page fault handler for userspace faults allows the CPU to schedule other tasks and serve interrupts while the handler is still running.

If the page fault handler can fix the fault, the previously interrupted task can continue at the saved instruction address. Fixing the fault is only possible if the fault was not due to some invalid access. In cases where it is not possible to recover from a page fault, the faulting process will be sent a signal corresponding to the type of erroneous access.

2.5 System Emulation with gem5

When designing new low-level system components, using system simulation is essential. While prototyping in hardware may be feasible under certain circumstances, using a software-based approach, like the gem5 simulator [Bin+11], allows for fast and cheap changes of all system components. With gem5, it is possible to emulate a complete computer system down to cycle-level accuracy. Instead of simulating each cycle, gem5 uses an event-driven system that only simulates the interaction between system components and thus allows for fast and accurate simulation.

At the core of each gem5 simulation are parameterized components or *simulation objects* (SimObjects). Each system is a combination of multiple interacting SimObjects arranged in a specific structure. By altering the structure, the parameters, and the SimObjects, gem5 allows for a large variety of simulatable systems and architectures. Internally, simulatable components are implemented as a combination of C++ classes and Python files. The C++ code implements the actual functionality of the component, while the Python file holds information on the SimObject and the parameters of the component. When building the simulator binary, the build scripts generate Python

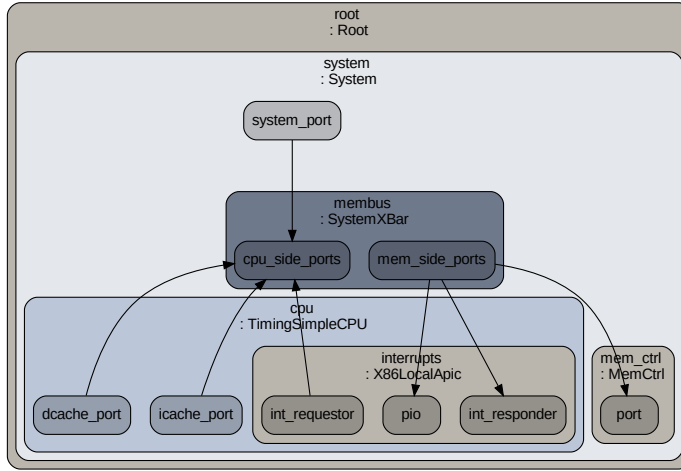


Figure 2.8: Structure of a basic gem5 system. The system consists of a CPU and a memory subsystem. The membus SimObject connects the CPU and the memory controller.

bindings for each simulatable component. These bindings expose the SimObjects and their parameters such that they can be used in the simulation.

When simulating a system, one has to provide a configuration script that interconnects and parameterizes the SimObjects. A minimal system consists of a CPU model and a memory system [Low20]. The memory system comprises one or multiple storage devices, and a bus interconnect that allows the CPU to access the storage. System setup is done through a Python configuration script passed as an argument to the simulator binary. All SimObjects generated by the build script are available in the configuration script. The configuration script describes the system as a hierarchical model of SimObjects with the **System**-object at the top. This system object defines shared parameters like common clock and voltage sources for all child objects. Figure 2.8 shows the components needed for a minimal gem5 system.

In this structure, the CPU's local APIC controller connects to the memory bus. The bus is, in turn, connected to the memory controller and the cache ports of the CPU. As the `mem_ctrl` component manages the DRAM as an internal component, and it is not depicted explicitly in this graph. The DRAM will still be simulated as a separate component that receives commands directly from the memory controller. With this minimal system, it is already possible to simulate workloads in the form of pre-compiled binaries by defining them as processes in the configuration. As the configuration script offers all basic Python functionality, it is possible to change the system structure even

during execution. This flexibility allows hot-swapping CPU models and other system components during execution.

2.5.1 Emulation Capabilities

When simulating more complex workloads, one can run gem5 in *full system* (FS) or in *syscall emulation* (SE) mode. The full system mode can provide cycle-accurate emulation of a complete computer system depending on the chosen system settings. In this mode, it is possible to boot a Linux kernel and execute programs like in a virtual machine. These capabilities allow prototyping of hardware-software co-designs as one can run a modified version of the Linux kernel on the simulated hardware. Once the full system simulation has successfully booted, one can attach an emulated terminal to the simulated system to interact with it.

Contrary to FS mode, the syscall emulation mode does not simulate a stand-alone operating system. Instead, the SE mode only simulates a given workload as a process and terminates after finishing the execution. As the given workload might perform syscalls and I/O-operations, interactions with the operating system are internally simulated by gem5. In SE mode, gem5 only emulates the CPU and the memory subsystem. While this might still yield sufficiently accurate results, the timing measurements and power traces will not be as accurate as in FS mode.

Due to the higher number of simulated components, full system mode is significantly slower than syscall emulation mode. Especially booting the guest operating system can take up a significant amount of time. One way to circumvent long waiting times is the use of checkpoints. Once an operating system finishes booting in FS mode, it is possible to create checkpoints for the simulator from the emulated shell. Such a checkpoint contains detailed information on the state of the complete system. When running a subsequent simulation, one can load the checkpoint and thus skip the tedious boot process.

Depending on the system configuration, the system can run in *timing*, *atomic* or *functional* mode. When executing the simulation in timing mode, all components use event queues to simulate transactions and resource contention. This mode yields the most accurate simulation results. It also imposes the highest simulation load on the host system as gem5 simulates multiple event queues in parallel. In atomic mode, system components immediately perform requested memory operations. Access timings are estimated on a per-component level, and the overall timings are not as accurate as in timing mode. Functional mode is used for system fast-forwarding and loading data from the host system to the emulated storage.

2.5.2 The Memory System

Apart from a CPU model, each emulated system needs to have a memory system. As depicted in Figure 2.8, a minimal memory system consists of a bus and a memory controller [Low20]. More sophisticated system models also include data and instruction caches and multiple bridges to other components. Each component in a system connects to one or multiple other components via *ports*. In gem5, we distinguish between request

and response ports. As the names suggest, request ports actively send requests to other ports, while response ports react to requests by sending the corresponding responses.

In *gem5*, components communicate with each other via *packets*. A packet is a basic unit of information sent via the port interface from one component to the other. In the memory system, each packet encapsulates a single object that is an instance of the `MemReq` class and additional metadata. `MemReq`-objects represent memory transactions from one `SimObject` to another. Such memory requests are issued by a requestor, wrapped in a packet, and sent over the memory bus to the corresponding receiver. During its lifetime, each memory request has a related memory command. This command changes throughout the transaction and holds information about the current stage of the memory transaction.

The ports of a memory object have to implement different access modes to accommodate the simulator's three execution modes. When sending a request in atomic mode, the response port returns an approximation of the time the memory access will take based upon fixed timing parameters. In timing mode, on the other hand, each access is simulated to its fullest, and ports may postpone responses if the memory device is busy. Hence, timing mode gives the most accurate timing results but is also the slowest access mode. Functional accesses are necessary if the memory object should allow fast-forwarding and direct loads. In general, it is not necessary to implement all three access modes for each memory component. Only the access modes that connected components will actually query are needed.

In the *gem5* memory system, the memory controller directly interacts with the DRAM. During system setup, one can select the exact type of DRAM that the system will use. Out of the box, *gem5* offers a variety of memory components ranging from DDR3 devices to LPDDR5 memory. The internal DRAM timings of all available models can be set freely in the configuration script, thus allowing for easy customization. When executing the simulation in timing mode, the memory controller emulates DRAM accesses to follow the signal order as described in Section 2.1. Read and write operations are queued in an internal access queue, and the memory controller reorders accesses to reduce the overall latency. Like in real hardware, reads are always served as burst accesses, and the result returned over the memory bus always fills an entire cache line. A series of discrete events simulate the internal DRAM state in the DRAM component connected to the memory controller. By transitioning from one state to another while adhering to the configured timings, the DRAM model yields an accurate timing estimation for all possible access scenarios.

2.5.3 CPU Models and the x86-64 ISA

Similar to the previously discussed DRAM models, *gem5* offers a variety of CPU models that can be used during simulation. The most basic CPU model is the *BaseSimpleCPU*. This model is an incomplete model that cannot run on its own but serves as a template for the *AtomicSimpleCPU* and the slightly more sophisticated *TimingSimpleCPU*. At its bare minimum, each CPU model needs to handle memory operations and offer a `tick()` function called every CPU cycle. The *BaseSimpleCPU* does not implement this `tick()`

function but provides functionality for common operations like advancing the program counter, checking for interrupts, or setting up instruction fetch requests. The basic CPU models are in-order systems, meaning that advanced CPU features like speculative execution and instruction reordering are not implemented.

When using the AtomicSimpleCPU, all memory requests are sent via the atomic interface of the connected ports. According to atomic simulation, memory operations will use latency estimations instead of the detailed hardware simulation. Average latency values are chosen to simulate stalls due to cache latencies. Due to the sequential nature of this model, each fetch-decode-execute cycle will take multiple ticks to complete.

In contrast to the atomic model, the SimpleTimingCPU uses the event-driven hardware simulation. In this more realistic simulation, read and write accesses can stall, and resource contention influences all latencies. Like real CPUs, the fetch and the execute step are split up and can happen in parallel. Once an instruction fetch has finished, the CPU executes the fetched instruction. For instructions that consist of multiple micro-ops, the CPU iteratively executes each micro-op. As there is no instruction or access reordering active, instructions will always be executed in the same order as they were enqueued in the fetch queue.

Both CPU types simulate page faults occurring during instruction fetching by explicitly checking for them after each fetch. Other faults that happen during the execution of the instructions are caught later on when the execution is simulated. Regardless of the fault source, the simulated CPU will invoke an architecture-specific fault routine that primes all affected CPU registers for the execution of the fault handler in the simulated system. As exceptions can occur when executing a micro-op, the simulator re-starts the complete instruction in such a case.

Contrary to the basic CPUs, the O3 CPU implements advanced features like branch prediction, speculative execution, and instruction reordering. This CPU model yields the most accurate results, as it closely resembles the function of a modern CPU. The O3 CPU implements the pipeline stages as described in Section 2.3. Like the TimingSimpleCPU, the O3 CPU schedules events in a queue. The simulator always executes the next possible event from any of the event queues. While this CPU model yields the most accurate timing results, it is also the slowest model. Due to the instruction reordering, the O3 CPU has to simulate significantly more operations than any other CPU model. Thus, this model should only be used when profiling relatively small workloads. In full system mode, booting with the atomic CPU and switching to the O3 model is a feasible approach.

The previously described CPU models do not implement a particular ISA. Instead, each ISA in gem5 is defined separately, and the CPU only calls common methods that simulate the implemented instructions. Concrete ISA implementations are split over multiple files that implement registers, low-level interrupt and fault handling, architectural traits, and instruction decoding and execution. As each ISA has its characteristic instruction format, each implemented architecture needs to define a class that handles instruction decoding. For the x86-64 ISA, this decoder class is implemented as a state machine that follows the instruction format described in Section 2.3. Instructions are defined via a combination of C++ code, Python files, and gem5-specific `.isa` files. Instructions are

grouped according to their opcode length and the `.isa` files provide decoder functions for each type of operation. During compile-time, the build scripts generate C++ decoder functions from the given instructions definitions. The previously mentioned decoder class then uses these functions to identify instructions from their architecture-specific encoding correctly. As each decoder model provides a common function interface to the CPU model, one can use arbitrary combinations of CPUs and ISAs during simulation.

The x86-64 ISA implementation shipped with gem5 implements the complete x86-64 ISA and parts of Intel's AVX extensions. Vector registers up to 256 bit are available, and other extensions can be added by modifying the existing instruction and register definitions.

2.6 Cryptographic Hash Functions and MACs

In cryptography, a hash function is a deterministic function that maps an arbitrary input to a so-called *hash* or *tag*. Given a binary input M of arbitrary size n and a tag T of size t , a hash function H can be defined as

$$H : \{0, 1\}^n \rightarrow \{0, 1\}^t \quad (2.5)$$

As n can be chosen arbitrarily, a hash function must handle an unbounded number of possible inputs. In general, the size of the tag will be smaller than the size of the input ($t < n$), thus a hash function is not directly invertible. Given that $t < n$ holds, it is clear that there must exist at least two inputs M_1, M_2 such that $H(M_1) = H(M_2)$ holds. Such a case is called a *hash collision* if $M_1 \neq M_2$.

A cryptographic hash function needs to fulfill 3 security properties to provide security against an adversary:

Pre-image resistance Given a tag T , it should be hard to compute an input M such that $H(M) = T$. This property ensures that an attacker cannot forge a message that fits a given tag. Given an optimal hash function, the probability that a message maps to one of the possible tags is $\frac{1}{2^t}$.

Second pre-image resistance Given an input M_1 , it should be infeasible to find a second input M_2 such that $H(M_1) = H(M_2)$. This criterion guarantees that an attacker cannot find a message that can be substituted with a given message without changing the resulting tag.

Collision resistance An attacker should not be able to compute $M_1, M_2, M_1 \neq M_2$ such that the inputs cause a hash collision. Contrary to the previous properties, the attacker can freely choose both messages when performing a collision attack.

These three properties define how an optimal hash function should behave. If the input and the output have the same size, an optimal hash function would essentially be equal to a one-way function. As it is impossible to implement such an optimal function,

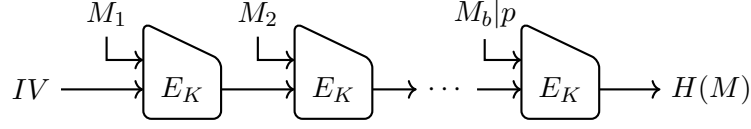


Figure 2.9: Merkle-Damgård iterated hash function

real-world implementations need to give a certain level of security that is guaranteed against generic attacks. The security of a hash function is quantified by the number of trial computations that an attacker needs to perform to break the security properties. With a tag size of t , it takes 2^t trials to compute a pre-image or to break the second pre-image resistance on average. Due to the birthday paradox, collision resistance is expected to be broken with $2^{t/2}$ trials. This fact entails that a hash function with a t -bit tag offers $t/2$ -bit security. Modern hash functions usually provide security levels of at least 112 bit.

Cryptographic primitives act as the building blocks of hash functions. One of the most common types of hashes are *Merkle-Damgård* (MD) constructions. In such a hash, the tag is computed by iterating compression functions with message blocks as their input. Additional padding bytes must extend the last message block. This extension hardens the MD-hash against length-extension attacks. Hashes created with this method base their security on the underlying cryptographic primitive. The collision resistance of an MD-hash is equivalent to the collision resistance of the internally used compression function.

Figure 2.9 illustrates the basic structure of a Merkle-Damgård hash. The message is split into b message blocks, and the last message block is padded with the message length. Each compression block compresses a message block and the output of the previous block. Intermediate values in the chain are called *chaining values* and they represent the *internal state* of the hash function. As the first block does not receive a chaining value, an *initialization vector* (IV) defines the initial state of the hash function. This arbitrarily chosen value must be the same for all communicating parties. The hash of the message M is the output of the last block in the chain.

Hashes that have an internal state size that is similar to the size of the final hash value are called *narrow-pipe constructions*. Contrary to that, a *wide-pipe construction* has an internal state that is much larger than the produced hash value. Lucks [Luc04] introduced these constructions to account for some of the vulnerabilities of narrow-pipe hashes. Increasing the internal size decreases the probability of collisions in the intermediate hash values, thus thwarting attacks based on internal collisions.

Cryptographic hash functions provide message integrity. Given a n -bit message M , one can compute the matching tag as $T = H(M)$. A sender transmits the message via an insecure channel, while the tag has to be transmitted securely. The message receiver can then re-compute the hash and compare the result to the tag received via the secure channel. A hash mismatch indicates a change in the message. Such a change

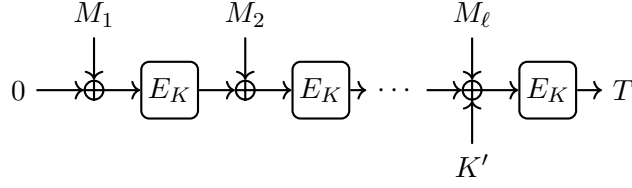


Figure 2.10: Schematic CBC-MAC construction

can be the result of transmission errors or deliberate tampering through a malicious man-in-the-middle. As hashes are unkeyed functions, they cannot provide authenticity.

Message Authentication Codes (MACs), on the other hand, guarantee authenticity as they compute the tag from a m -bit message M and a secret k -bit key K . Similar to hashes, a MAC might be constructed from a variety of cryptographic primitives. MACs built upon hash functions are called *hash-based message authentication codes*, or HMAC. A MAC H_K can be described as a function

$$H_K : \{0, 1\}^m \times \{0, 1\}^k \rightarrow \{0, 1\}^t. \quad (2.6)$$

Krawczyk et al. [KBC97] define the HMAC function as

$$H_K(M) = H((K \oplus o) || H((K \oplus i) || M)). \quad (2.7)$$

In this formula, o is the outer padding, and i is the inner padding. Given that H is an iterated hash that hashes message blocks of B -byte size, the outer padding is a block filled with `0x5c` and the inner padding a block filled with `0x36`. Keys smaller than the block size are padded with zeros, and larger keys are hashed before use. As the HMAC construction builds upon an iterated hash function, it is vital for the security of the HMAC that the underlying hash function is secure.

Another possible MAC construction is the so-called CBC-MAC. CBC stands for *cipher-block-chaining* and describes the basic structure of such a MAC. Figure 2.10 shows how block-cipher primitives can be chained to compute a MAC of a message M . The message is split into blocks that fit the input size of the block-cipher primitive. This size depends on the specific cipher in use. If the message length is not a multiple of the input size, the last block will be padded accordingly. A CBC-MAC needs two keys K, K' , and an initialization vector. Without loss of generality, the IV in the figure is set to 0. To compute the MAC, we iteratively encrypt each message block and add the output of the previous block to the following message to encrypt. In the penultimate encryption, the second key K' is additionally XORed to the input of the block-cipher. Using a different key in this step secures the MAC scheme against length extension attacks. The actual tag is the output of the last block-cipher.

While there is a variety of possible MAC constructions, all constructions share the same security property. To be considered secure, a MAC needs to be *unforgeable*. Given an instance of a MAC with a fixed key K , it should be infeasible for an attacker to produce a new M, T -pair such that $H_K(M) = T$ holds. Due to the name of the security property, attacks on MACs are called *forgery attacks*.

Chapter 2 Background

MACs protect the authenticity of a message. By using a shared, secret key, the tag and the message can be transmitted via the same insecure channel. Contrary to transmission with a hash function, the challenge when using a MAC is keeping the key secret. Given that the key is only shared between the communication partners, unforgeability guarantees that even if the attacker can modify the message, it is not possible to predict the tag such that it is valid for the modified message. Thus, the receiver can register if a malicious party corrupts the data.

Chapter 3

Design Approach and High-Level Overview

In this chapter, we discuss the principle design of the proposed integrity protection approach. Section 3.1 defines the attacker’s capabilities and gives an according threat model. Section 3.2 provides a high-level overview of the complete mitigation and elaborates on which system components are involved. In Section 3.3 we discuss how MACs can be used to protect the integrity of DRAM data. Finally, Section 3.4 elaborates on how data correction is moved from hard- to software.

3.1 Threat Model

As discussed in Section 2.2, the majority of modern DRAM devices experience single- or even multi-bit errors under certain circumstances. While naturally occurring bitflips may cause random data corruption and system crashes, maliciously induced bitflips have the potential to compromise system security deliberately. Due to the pervasiveness of personal computing devices and modern DRAM technology, it is necessary to protect against attacks based on Rowhammer. Unfortunately, most existing mitigations were proven to be ineffective and state-of-the-art ECC-DRAM cannot protect against sophisticated Rowhammer attacks.

When designing novel types of integrity protection, it is not sufficient to solely consider naturally occurring bit faults. Such events are relatively rare, and existing integrity protection measures like ECC mitigate them sufficiently. Instead, the security requirements of the system should be fit against the strongest known type of attacker. Existing research and attacks using Rowhammer suggest that one needs to assume that an attacker can arbitrarily induce bit faults in DRAM. To model existing attacks even more precisely, we need to anticipate that an attacker can induce multi-bit faults with reasonable precision. With this assumption, existing integrity protection measures like ECC with Hamming codes cannot protect against the attacker as maliciously induced multi-bit errors remain undetected and cause silent data corruption [Coj+19]. Furthermore, we need to assume that the attacker has at least knowledge of the unmodified data. Under this attacker model, the potential threats are:

Data Corruption By flipping bits in memory, the attacker can arbitrarily alter stored data. This corruption stays unnoticed by the victim.

Denial of Service Modifying program data can lead to invalid opcodes, control flow violations, and other unwanted side effects. If an attacker can modify loaded code, it is possible to crash running processes. On server or cloud computers, Rowhammer attacks can potentially lead to large-scale outages.

Privilege Escalation As described in Section 2.2.1, existing attacks demonstrate that Rowhammer can be used to gain kernel privileges. An attacker that can arbitrarily induce bitflips can manipulate the kernel’s paging structure or privileged executables like `sudo` and, thus, perform a privilege escalation attack.

Silent Data Corruption Additionally to error detection, data integrity protection schemes aim to correct bit errors. Under certain circumstances, the error correction may fail and correct the data to a value that is not equal to the untainted value. While such situations are not predictable by the attacker, they are still a potential threat that needs to be considered.

Based on existing research, we can estimate how many bitflips per row the attacker can realistically induce. In their 2014 work, Kim et al. [Kim+14] found that the worst tested DRAM module had up to 4 bitflips at once. Thus, we must assume that an attacker may, at least, induce this number of bitflips in a single row.

With this attacker model, we can evaluate the effectiveness of our mitigation. The mitigation is effective if the attacker cannot reach one of the above goals within realistic constraints. In detail, this means that the attacker must not be able to construct a bitflip pattern that the mitigation cannot detect or that causes silent data corruption.

3.2 General Overview

To mitigate an attack with the given capabilities, we propose a novel hardware-software co-design that protects against arbitrary multi-bit errors. According to the classification scheme mentioned in Section 2.2.2, our proposed mitigation is elimination-based as it prevents the CPU from ever using faulty data. Figure 3.1 illustrates the principle components of the mitigation and how they interact with each other.

MAC-based integrity protection acts as the core of our mitigation. Like in conventional ECC solutions, the DRAM module offers an additional chip that stores computed checksums. While ECC uses linear codes that offer a rather low Hamming distance, our mitigation uses a MAC computed over the stored data. As the selected MAC is highly non-linear, multi-bit errors can be detected easily. Each data word is 512 bit large, and each MAC comprises 64 bit. This yields a similar distribution of data and checksums as a regular (72,64) Hamming code. On each memory access, the memory controller computes the MAC. This computation is implemented in hardware to minimize latencies. Write accesses will always update the stored MAC but do not validate the previously stored data. On read accesses, the memory controller fetches the data and the corresponding MAC for validation. As the bus size for ECC-DRAM modules is 72 bit, the memory controller loads 8 data bytes and one checksum byte simultaneously

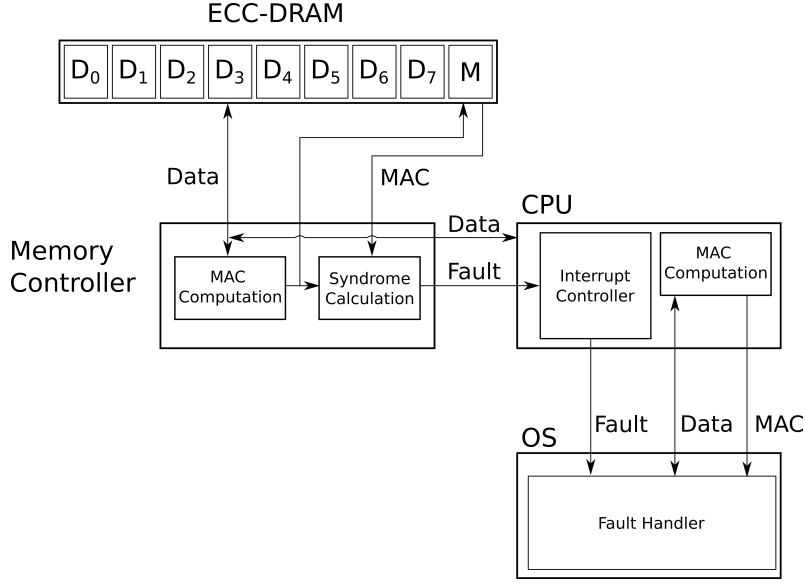


Figure 3.1: High-level overview of our proposed mitigation. A commodity ECC memory module stores the computed MACs. On each memory request, the content of the DRAM is validated using the stored MAC and a freshly computed MAC. A mismatch will trigger an exception in the CPU.

from the DRAM. Due to the larger data-word size, it is necessary to load additional data from memory as the MAC is always computed over a complete 512-bit block. Note that this does not increase latency as accesses will always happen in burst mode, and the data word size is equivalent to the size of a cache line. By comparing the stored MAC with the freshly computed MAC, the memory controller computes the syndrome. A non-zero syndrome indicates that memory corruption has taken place and error correction is necessary. Syndrome computation is also implemented in hardware in the form of a low-latency bitwise compare circuit. To avoid stalling on load operations, MAC validation is done on the fly while the data of the block is loaded. This independent computation leads to the advantage that the CPU can continue transient execution with the loaded data. In the case of no bitflips, this design does not impact timing performance as it acts completely transparent. On a MAC mismatch, the memory controller informs the CPU by raising a special fault and thus jumping to the corresponding fault handler.

In the fault handler, the operating system tries to correct the corrupted data by iteratively flipping bits and computing the MAC. As bitflips in memory can manifest in the data and the stored MAC, the error correction algorithm does not distinguish between data and MAC when trying different bitflip patterns. The CPU implements a novel instruction in every core that performs MAC computation like in the memory controller to speed up error correction. This instruction is an extension of the AVX-512 instruction set. Using the 512-bit ZMM register for data and an additional 64-bit register for the

MAC, the novel instruction computes the MAC of the provided data and compares it with the MAC passed in the 64-bit register. On match, the CPU's zero flag is set, indicating a successful correction. As it is impossible to determine the number of flipped bits, the correction algorithm iteratively tries all one-bit faults before testing for multi-bit errors. To correct the data, the CPU needs access to the data and the MAC without triggering the fault again. For this, the memory controller provides two special address spaces that the operating system can map. The *raw data space* allows accessing corrupted data without MAC validation, thus avoiding subsequent faults. As the correction algorithm also needs access to the stored MAC, the memory controller provides a *tag space* that has the same layout as the raw data space but returns the corresponding MAC on access. These address spaces are only accessible from kernel code. Userspace processes that try to interact with these ranges will cause a fault and subsequently be terminated by the operating system.

By shifting error correction from hardware to software, our proposed mitigation offers a flexible error handling method. The operating system can group memory in protection classes where a stronger protection class means that more potential bitflips can be corrected. As error correction may take longer for scenarios with multi-bit errors, the operating system may restart the corrupted process or halt the system in the case of errors in the kernel.

As bitflips may occur at any location, the fault handler itself can become corrupted. Such a scenario would lead to an unrecoverable situation. Hence, we propose that the fault handler is implemented redundantly. By placing multiple fault handlers with the same code but different vectors in the IDT, we harden the system against attacks on the correction mechanism. Each time a fault due to data corruption occurs, the executed handler is selected in round-robin order. Thus, even in the case of data corruption in one or multiple of the handlers, the other handlers can correct the faulty data as long as at least one handler is unaffected.

3.3 Protecting Integrity with MACs

When computing checksums over data, there are several types of mathematical functions to consider. As described in Section 2.2.3, commodity ECC tags offer insufficient protection against Rowhammer attacks as they are linear. Using simple hash codes for the tag computation leads to better security than the linear ECC tags but still has the disadvantage that an attacker can try to find offline hash collisions. Such collisions again compromise the security of the system as the attacker would be able to alter data while staying undetected by the mitigation. Protecting the DRAM with MACs eliminates this drawback of hashes. As MACs act like keyed hash functions, a MAC's collision probability with a fixed key is equivalent to that of a hash of the same size. While hashes can be computed without any additional data, MAC computation requires a key. This key is unknown to the attacker, implying that offline attacks become infeasible. In our proposed mitigation, the key for MAC computation is generated once when booting the system. It is then stored in each CPU core and the memory controller. Our design offers

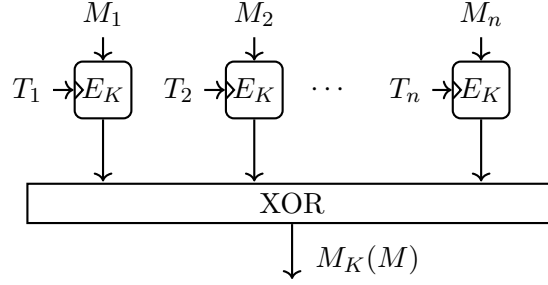


Figure 3.2: Structure of the MAC used to protect data integrity

no interface to read from or write to the key as it will never be used outside the hardwired MAC computation. As the randomly generated key needs to be cryptographically secure, we propose to use the same mechanisms as Intel’s RDRAND instruction for key generation. According to the Intel Software Developer Manual [Int21b], this instruction provides randomly generated numbers conforming to the NIST standard [BK12]. Thus, using the same number generation strategy yields truly random keys. As described above, each 512-bit data block acts as the input for the corresponding MAC computation. Each MAC has a size of 64 bit. Thus, a block consisting of data and MAC is 576 bit in size. Protecting the stored data with MACs allows us to think of Rowhammer attacks on the mitigation as if the attacker tries to forge a valid MAC. As described in Section 2.6, MACs are constructed to be unforgeable under chosen-plaintext attacks. In the attacker model formulated in Section 3.1, we assume that the attacker has knowledge of the data before inducing bitflips. With the mitigation in place, reading the modified data is not possible. Also, unlike in the cryptographic security proof, our design does not permit the attacker to read any MAC, as they are never mapped in an accessible address space. Thus, in the proposed scenario, a forgery attack on the MAC is even harder as the attacker can only approximately modify the plaintext and has no knowledge of the stored MACs.

Every memory access will cause a MAC computation. Hence, this step must be done as quickly and efficiently as possible. MACs built on iterated block ciphers are unsuitable for our mitigation. As the memory controller always fetches the critical word before loading the subsequent blocks of the 512 bit data word, the computation will be delayed in certain access patterns. Instead, the MAC is implemented as the sum of 8 independent tweakable block ciphers as depicted in Figure 3.2. Each block cipher produces a 64 bit intermediate value that is summed with the other intermediate values by an XOR block. With this construct, the MAC can be computed on the fly as the memory controller receives the data packets of the burst access.

All block cipher instances take the data, the shared key, and a unique tweak as input. When computing the MAC as described, it is essential to use a different tweak for each data block. Failing to do so will result in a MAC that is not forgery resistant as blocks with the same tweak and data will cancel each other out. Instead of using random tweaks,

the mitigation uses the index of the data block as the tweak during computation. The critical word always has an index of 0, while all subsequently loaded words of the burst increase this number by 1. Note that this MAC is not unforgeable given that the attacker can read the resulting MAC. Under our proposed threat model, the attacker has no way to read the MAC. Thus, our MAC construction is unforgeable for an attacker that is limited to Rowhammer-type attacks.

Upon read access, the stored MAC will be validated against the freshly computed MAC. Each MAC is 64 bit in size. Thus, it takes a complete burst to load the entire MAC. This only slightly influences the overall time needed until validation is complete. Once the tag for the loaded data is computed, both MACs are compared. The syndrome is computed as the number of bits in which they differ. When no data corruption has occurred, this difference is always zero.

Other proposed DRAM integrity protection schemes also use MACs as tags for the stored data. Huang et al. [HS10] propose IVEC which uses GMAC to compute the tags. Much like the in-band ECC protection mentioned in Section 2.2.3, IVEC stores the tags next to the data in DRAM. While this allows usage with non-ECC DRAM, it also increases the latency of memory accesses as an additional transaction between the memory controller and the DRAM is necessary. Synergy, a scheme developed by Saileshwar et al. [Sai+18], uses ECC DRAM to store the computed tags. Additionally, they store parity bits in DRAM to increase the correction capabilities of their method.

3.4 Error Correction in Software

Contrary to other integrity protection methods, our mitigation shifts the error correction step from hardware to software. While conventional error correction in hardware is limited and inflexible, the software-based approach allows for adjustable and extendable error handling. Unlike linear codes, MACs do not indicate the location of the error in the faulty data word. Hence, correcting MAC protected data is an iterative process. According to the attacker model defined in Section 3.1, MAC corruption is also a possibility. Thus, the correction algorithm needs to consider 3 types of error:

Data Corruption If bitflips manifest exclusively in the 512 data bits associated with the stored MAC, the computed tag will significantly differ from the stored value. For optimal hash functions or MACs, each input bit influences approximately half of all output bits.

MAC Corruption In cases where the attacker-induced bitflips only affect the stored MAC, the computed syndrome is expected to be a small, non-zero value.

Corruption of Data and MAC When both, MAC and data, are compromised, it is hard to precisely predict the impact on the MAC. In the most probable scenario, the computed MAC will differ from the stored MAC by more bits than in the case of pure

MAC corruption but not as strongly as in the case of data corruption. This case is hard to distinguish from pure data corruption and poses the worst type of bitflip pattern.

In the fault handler, the correction algorithm needs to access the raw data and MAC. As described in Section 3.2, we introduce two new address spaces to the memory controller. By mapping these two address ranges, we grant the fault handler direct, unvalidated access to the DRAM contents.

As a first step in the correction procedure, the algorithm needs to distinguish between the three cases. During the computation of the syndrome, the memory controller determines the number of differing bits between the two MAC values. This number is equal to the Hamming weight and is used to decide whether the fault is a pure MAC corruption or any of the other cases. The threshold number of wrong bits is a tunable parameter that can be set in the kernel. Setting this number too high has a negative impact on the security of the system as data corruption might go unnoticed, while setting the threshold too low can result in performance loss.

Should the number of faulty MAC bits be sufficiently low, the fault handler only needs to write the data back to DRAM. This updates the MAC and restores the correct value. Pure MAC faults constitute the best-case scenario, as no additional MAC computations are invoked.

In the other two error cases, the correction algorithm checks if the compromised data is file-backed and can be restored by reloading. This approach works for all types of error but is unnecessary in the case of pure MAC corruption. The actual correction loop needs to be invoked for data that cannot be restored from the file system.

This loop aims to find a bitflip pattern with minimal Hamming weight that generates the correct MAC when applied to the data. Single-bit errors are the most probable faults that an attacker can induce. Thus, the correction algorithm first tries all possible bitmasks where one bit is set. If none of the bitmasks leads to the correct result, the algorithm increases the Hamming weight of the bitmask by one. The maximum Hamming weight is a configurable parameter that depends on the protection class of the faulty memory area. It is equivalent to the maximum number of correctable bits. A larger Hamming weight can be used for system-critical memory that needs to be restored even in the case of multi-bit errors. Memory for user processes and other non-critical data can allow a smaller number of correctable bits. Once the algorithm finds the correct bitmask, the loop terminates and writes the corrected data back to DRAM.

As the correction loop tries all possible bitmasks of a certain Hamming weight before moving on to the next stage, the runtime of the correction strongly increases with the number of possible bits to choose from. Thus, discerning between pure data corruption, MAC corruption, and the combined case helps to improve performance. The best-case of faults is the case in which only the MAC is affected. As the handler only needs to write the data back to DRAM, no additional operations are performed, and the correction routine does not need to map the tag space. When only the data is faulty, all bitmasks are applied to the 512 data bits, significantly reducing the maximum search space. In contrast, the combined fault is the worst case for the proposed error correction algorithm. There, the search space measures the full 576 bits as each bit is potentially faulty.

Chapter 3 Design Approach and High-Level Overview

While the proposed software-based correction is slower than hardware implementations when correcting single-bit errors, it offers much more robust error correction capabilities. For systems that are hard to maintain or where termination on multi-bit errors is unacceptable, commodity ECC solutions are insufficient.

Chapter 4

Hardware Modifications

In this chapter, we discuss the hardware modifications required to implement our proposed mitigation. The memory controller has to be modified such that fast MAC computation and validation are possible. This is discussed in Section 4.1. In Section 4.2, we elaborate how the AVX instruction set is extended to include the novel instruction used in the correction loop.

4.1 Modifying the Memory Controller

As the memory controller handles all memory requests, it plays an essential role in the mitigation. While the basic functionality of the memory controller stays untouched, some additions are necessary to implement the integrity protection. We assume that the underlying memory controller has a data bus width of 72 bit and thus can use ECC-DRAM modules. This precondition is necessary as we want to avoid the additional loads that are necessary with in-line designs. As ECC-DRAM modules are commodity products and widely used in server computers, this assumption does not limit the applicability of the mitigation.

4.1.1 Validation Logic

Upon read or write access, the memory controller validates the complete data block containing the requested address. With 512 bit per data block, each memory access thus validates a complete cache line. Choosing this specific block size yields the advantage that a single burst is enough to access a complete data word. When performing a burst access, the memory controller reads eight columns from the same row in quick succession. Usually, the first column read is the so-called *critical word*, which is the data the read operation initially requested. Loading these eight bytes first allows the CPU to continue transient execution while the rest of the burst is still being processed. After the initial bytes are loaded, the rest of the bytes in the data word are accessed. As we use ECC-DRAM modules with a 72-bit wide data bus, each access in the burst is accompanied by a part of the stored MAC. As described in Section 3.3, the MAC is computed on the fly. Each 8-byte word is used as an input for the corresponding block cipher circuit during the burst. The block cipher in use is assumed to handle 64-bit wide data input, and each instance produces 64 bits of output. Apart from that, there are no constraints on what block cipher to use. As each encryption operation needs to access

the shared key, this value is stored on the memory controller to keep latencies low. For this, a single additional register with the desired key size is added to the controller. This register is implemented as a read-only register that is set once during booting.

After finishing a burst access, the MAC circuit has completely computed the MAC of the data. The syndrome computation circuit compares the computed MAC against the MAC that was loaded during the access. The syndrome is computed as the difference between the expected and the actual value in commodity ECC circuits. While such an approach would also work for our mitigation, we instead compute the number of set bits in the difference between the two MACs. This value is called the *popcount* and can be implemented in hardware as well. Computing the popcount allows us to pass the number of faulty MAC bits as an error code to the exception generated in the CPU. A non-zero popcount signals a mismatch between the two MACs and indicates that any of the three corruption cases occurred. Such a case causes the CPU to raise the novel exception and initiate the error correction.

4.1.2 Simulator Modifications

In the gem5 simulator, the memory system implements the memory controller such that the DRAM in use is a parameter of the memory controller. This design allows exchanging DRAM models while using the same controller easily. As the standard memory controller of gem5 does not support ECC-DRAM, it is necessary to extend its functionality such that it can handle the 72-bit data bus. To provide the raw data space and the tag space to the operating system, the extended memory controller announces these memory ranges in addition to the actual memory range of the DRAM. The additional memory ranges are mapped to physical addresses constructed by combining the actual physical addresses of the DRAM with pre-defined bitmasks. Setting bit 41 in the physical address identifies addresses belonging to the tag space, while bit 42 signals that an address is part of the raw data space. As the ninth chip of ECC-DRAM modules is reserved for the tag, ECC-DRAM modules only allow $\frac{8}{9}$ of their available capacity to be accessed by the system. The MAC computation and validation circuits are deeply rooted in the extended memory controller. On write access, the MAC of the complete cache line is computed and stored in multiple columns on the additional chip. Each burst read access fetches the parts of the MAC from the chip and compares the overall MAC with the freshly computed MAC of the read data. In the simulator, the MAC computation circuit waits for all packets of the burst.

Following the description in Section 2.5, components in the memory system communicate with each other via packets. The extended memory controller uses this communication interface to notify the CPU in the case of a checksum mismatch. Each CPU instruction that performs load and store operations might potentially trigger a checksum mismatch.

To model the latency of the MAC computation, each packet that passes through the memory controller is delayed by a configurable amount of time. This delay depends on the actual MAC implementation and the latency of the logic components that handle the syndrome computation.

4.2 CPU Modifications

As our mitigation is a novel hardware-software co-design, it modifies the CPU. These modifications do not introduce any breaking changes but extend the available functionality to accommodate our mitigation needs.

4.2.1 Instruction Set Extension

During the error correction, the CPU performs many MAC-and-compare steps in quick succession. We propose a novel instruction that implements the MAC computation in hardware as a separate execution unit to speed up this process. The low-level implementation of the MAC computation is functionally equivalent to the implementation in the memory controller. Adding this type of instruction to the CPU is necessary, as computing the MAC in the memory controller circuit would increase interface complexity and cause additional latencies. Adding the new execution unit to the CPU allows the core to execute the proposed **VCALCH**-instruction. The name of the instruction indicates that it is a type of vector instruction that computes a keyed hash over the data. We propose this naming following the AVX scheme, as the instruction extends the AVX-512 vector instruction set. When executed, the instruction uses two registers to pass parameters. The first register is one of the 512-bit ZMM registers. During the error correction algorithm, this register holds the data of the corrupted data word while a 64-bit register stores the MAC. This is a significant difference from the standard AVX instructions, as VCALCH mixes the types of used registers. Regular AVX instructions provide multiple operand modes. In the case of our new instruction, we only need to consider the operand mode with two registers, as all other modes would implicitly cause additional load requests. Calling the instruction triggers the MAC computation and validation in the CPU core. The execution unit computes the MAC of the data provided in the ZMM register. The result is then compared to the MAC given in the 64-bit register. If both values match, the zero flag in the status register of the CPU core is set.

4.2.2 Other Hardware Modifications

Similar to the previously mentioned key storage in the memory controller, the CPU is extended by an additional register that holds the key and is set once during booting. This key is accessed at each MAC computation and thus must be stored close to the validation circuit. Following the description in Section 3.4, faults occurring in the code of the fault handler also need to be corrected. We extend the CPU such that an internal counter keeps track of the currently executed fault handler. This counter wraps around to zero once it reaches the number of available correction fault handlers. When issuing the execution of the interrupt handler, the counter acts as an offset to the base interrupt vector such that the correct IDT entry is selected. Resetting the counter is unnecessary, as each fault handler is identical, and thus it is irrelevant which handler is executed. With this design, the CPU can call the fault handlers in a round-robin fashion.

None of the additional registers offers an interface with which they can be read from software.

4.2.3 Simulator Modifications

As discussed in Section 2.5, the `gem5` simulator defines a variety of instructions and ISAs. The simulator version used in this work implements only a small subset of vector instructions. During the error correction, the operating system performs mathematical operations on the 512-bit data word. Thus, adding the missing AVX instructions to the simulator is necessary. Fortunately, the `gem5-avx` [Wan20] project implements most of the missing vector instructions and can easily be integrated into an unmodified `gem5` version.

The standard AVX-512 instruction map is not completely populated with instructions. Thus, some of the possible opcodes are reserved for future use. Our novel instruction uses a two-byte opcode, and we place it next to the `VMOVDQ` instruction family. In general, every free opcode is a suitable identifier for newly added instructions. We emulate the MAC circuit in the CPU by calling the same MAC function used by the memory controller.

In the main loop of the simple CPU simulation, the CPU executes each instruction sequentially. Exceptions that occur during execution are detected by checking the return value of the instruction. When the memory controller detects a checksum mismatch, it sets the response type of the faulting packet to an error type. The CPU checks the packet type after each access and raises an exception if necessary. We implement the CPU exception similar to a page fault, the main difference being that our exception stores the physical address in the CR2 register.

Chapter 5

Software Modifications

In this chapter, we discuss the software modifications in the Linux kernel. We will elaborate how an exception handler for a new fault type is added to the pre-existing exception handlers in Section 5.1. In Section 5.2 we will give a detailed explanation of the implemented error-handling routine.

5.1 Introducing a new Fault Type

As the previously discussed memory controller modifications give rise to a new type of fault, the Linux kernel needs to provide a set of corresponding fault handlers that perform error correction. Each CPU fault causes a lookup in the IDT and subsequent execution of the handler function. Thus, IDT entries that point to the redundant fault handlers for error correction need to be added to the IDT structure as fault gates. All fault and interrupt handlers are declared using variations of the `DECLARE_IDTENTRY`-macro. The IDT struct is set up during booting. Hence, the error-correcting handler will only be available once Linux has finished the IDT initialization during startup. Adding the handler as an early interrupt handler is not an option as we require certain OS functionality that is not available until the system has properly booted. Not protecting against attacks at boot-time does not weaken the defense against Rowhammer attacks, as we can assume that the attacker can not effectively hammer the memory during booting.

Conforming to the previously discussed design of the new fault type, the entry stub of the exception handler assumes that the CPU pushes the syndrome as the error code on the handler stack. Thus, the stub places no additional dummy error code on the stack. For the new handler, the entry stub is equivalent to that of the regular page fault handler.

Due to the limited size of the IDT and the occupied fault handlers, we can place at most 10 instances of the redundant fault handler in the IDT. This amount of handlers should be enough to guarantee that at least one handler is not corrupted and thus able to correct all other handlers. As bitflips may be induced in all parts of the handler code, it is necessary that the handler code inlines all calls to other functions. This is the only way to avoid situations in which a bitflip in a kernel function disables all handlers at the same time.

5.2 Error Correction in the Linux Kernel

Regardless of the used gate, the error correction handlers always execute the same code. The error correction handler accesses floating-point registers, as our newly introduced instruction uses the ZMM register. Regular kernel code usually does not use these floating-point registers. Saving and restoring all FPU registers on each context switch takes a lot of time and is unnecessary for processes that do not use them in the first place. Instead, the scheduler checks for each process if the FPU registers are unused. Only processes that use these registers store them when performing the context switch. When re-scheduling a previously executed process, the `TIF_NEED_FPU_LOAD` signals the scheduler code if it needs to restore the FPU registers. This behavior complicates the proposed exception handler, as we need to access FPU registers when performing the error correction using the `VCALCH` instruction. Mismanaging these registers may result in data leakage or corruption of userspace data. One can consider 2 solutions for this kind of problem:

Non-preemptable execution No other process may be scheduled and executed on the same core when executing the error correction code. This approach essentially locks the core to the correction procedure and thus avoids that other processes using floating-point registers interfere with the correction. Linux provides the `kernel_fpu_begin` and the `kernel_fpu_end` function to allow for FPU use in kernel code. By disabling all sources of preemption for the calling process, these methods guarantee that the kernel code has exclusive access to the FPU registers. Also, the previous register states are stored and restored, respectively. Depending on the runtime of the correction, locking a core might decrease system performance significantly. Thus, this method is unsuitable if the exception handler should be interruptable.

Store and clear registers on context switch The main issue with context switches from the kernel code to some userspace process is that processes only store their FPU state if it was touched at least once [BC05]. This behavior leads to a problematic scenario when the kernel switches to a userspace process that has not used the FPU registers yet. As the registers are not automatically saved and restored, the faulty data is leaked to the process. Subsequent FPU use of the process invalidates the kernel FPU state, as it was never saved and, thus, is not restored when switching back to the kernel. One possible workaround for this problem is the introduction of a flag that tracks if the core has started to perform the error correction routine. The scheduler checks this flag and, if set, causes the scheduler to store at least the FPU registers that the kernel code uses on each context switch. With this approach, the exception handler can be interrupted anytime without risking the corruption of its FPU state. The major downside of this approach is that it reduces performance by storing FPU data checking if the process would ever interact with the data. Thus, the overhead of context switches for non-FPU processes is increased. When comparing the two approaches described above, the non-preemptable solution seems like a more robust implementation that requires less modification of the

existing kernel code. Thus, the reference implementation of the mitigation described in this work locks the core to the correction handler and resets the used floating-point registers after completing the handler.

Independently of the selected approach, the core logic of the error correction handler remains the same. Algorithm 1 shows the pseudo-code of the most basic variant of the error correction loop. In the simplest case, the exception handler does not use any of the hints provided by the error code and corrects the errors solely by brute-forcing bitmasks of increasing Hamming weight.

Algorithm 1 The pseudocode of the basic error correction loop. It iteratively tests all possible bitmasks and updates the faulty data once the correct mask is found.

```

1: Get physical address from CR2 register, raw data and raw tag
2: Disable preemption
3: Check if already corrected
4: Lock correction for current address
5: while not corrected do
6:   Generate next bitmask
7:   Apply bitmask to data/tag
8:   Use VCALCH to check if corrected
9:   if corrected then
10:    Write data back to DRAM
11:   end if
12: end while
13: Clear vector registers
14: Enable preemption
15: Return from Interrupt

```

As a first step, the algorithm needs to save the contents of the CR2 register on the stack. This register contains the physical address that caused the fault and is updated on each subsequent fault. Bitflips in the correction handler cause a fault each time the handler is executed. By using the redundant handler approach, we can correct such faults. When another handler corrects a faulty handler, the fault that causes the invocation of the second handler overwrites the content of the CR2 register. Thus, each handler must save this register. The CR2 holds the physical address that caused the fault. There are no guarantees that this address is aligned to the 512-bit data word size. Hence, the algorithm has to align the physical address to the address of the data block that contains the address of the corrupted data.

With the physical address of the data word, we map the two special address ranges into the virtual memory of the correcting process. These two ranges provide access to the faulty data and MAC without triggering additional faults on access. As described above, the correction locks the core it runs on as it must use the FPU.

The check in line 3 of the algorithm catches an edge case of the redundant handler approach. If the initially executed fault handler is corrupted and faults before storing the

content of CR2, this content is overwritten. Once the fault handler is repaired, execution continues, but the value of CR2 is not restored. Losing the CR2 content leads to a situation where the fault handler might try to correct data that is not faulty at all. Thus, the additional check allows the algorithm to terminate the fault handler and resume execution of the instruction that triggered the initial fault. This instruction will again trigger a fault that the repaired fault handler can then handle.

In a multi-core setup, two cores may access the same physical memory range in quick succession. Such access patterns can result in situations where both processes try to correct the faulty data block. To avoid conflicts caused by simultaneous correction, a data structure in the kernel holds a list of all physical addresses that the handlers currently correct. This data structure allows each process to stop other processes from correcting the same chunk of memory. Once one of the correcting processes finishes, the waiting process is informed and can directly return from the fault handler.

When a process reaches the main correction loop, it iterates through all feasible bitmasks and tests if they produce a correct result. The maximum number of set bits determines the runtime of the correction loop and is a configurable parameter. The order in which the bitmasks are computed is irrelevant for the success and the average runtime of the algorithm. If all bit permutations with a certain number of set bits fail to produce a correct result, the algorithm increases the number of bits and continues.

Clearing the vector registers before returning from the fault handler avoids leaking the data and the MAC through the registers used during the correction. The previously stored FPU state is restored in preparation for the handler exit.

Once the correction loop has been completed, execution resumes at the instruction that initially triggered the fault.

Chapter 6

Evaluation

In this chapter, we evaluate our proposed mitigation with respect to different characteristics. Section 6.1 discusses the setup of the simulated evaluation system. In Section 6.2, we analyze the security of the mitigation against the proposed attacker model. Section 6.3 discusses the performance and the runtime of the correction mechanism for different scenarios. Lastly, Section 6.4 elaborates how we measure the performance impact of the mitigation by running benchmarks on the simulated system.

6.1 Evaluation Setup

For all tests and benchmarks, we use one of the two system configurations listed in Table 6.1.

In the reference implementation, only the AtomicSimpleCPU and the TimingSimpleCPU support the novel interrupt type. Thus, testing the kernel modifications is only possible under one of these two CPU models. As we expect the timing model to yield more accurate results, we simulate all benchmarks regarding fault handling using System 1. System 2 uses the gem5 implementation of the out-of-order CPU. This CPU model yields the most accurate timing results, as it simulates instruction reordering and parallelism. Benchmarks that measure the performance impact of our mitigation in the error-free case are done using System 2, as the influence of the MAC latency on instruction reordering is only evident when using the O3 model. In the configuration of System 2, we use two values for the MAC latency. The zero-latency configuration is the reference system that simulates memory accesses as if no mitigation is in place.

While we do not limit our MAC design to a certain block cipher family, we use the QARMA_{5-64- σ_0} block cipher [Ava17] as a reference for our implementation. The QARMA block cipher family is designed to minimize latencies and the area of the hardware implementation. According to the authors, the latency-optimized version of QARMA used in our reference design has a latency of 2.2 ns. We include this latency as a timing parameter in the gem5 simulation and compare the overall runtime for different workloads. This latency does not contain the overhead for the XOR computation of the computed and stored MAC. Thus, we increase the overall latency in the simulation to 3.5 ns to account for all additional delays.

Table 6.1: System configuration of the benchmark systems

	System 1	System 2
Architecture	x86-64	x86-64
CPU Type	TimingSimpleCPU	DerivO3CPU
CPU Clock	1 GHz	1 GHz
Memory Type	2 GB DDR4_2400_8x8	2 GB DDR4_2400_8x8
MAC Latency	3.5 ns	3.5 ns / 0 ns
L1 Cache	64 kB L1i, 32 kB L1d	64 kB L1i, 32 kB L1d
Last Level Cache	2 MB	2 MB
gem5 Mode	Full System	Syscall Emulation
Linux Kernel	Modified Kernel version 5.9.2	Host Kernel / Emulation

6.2 Security Margins

By basing the security of our mitigation on the well-studied problem of MAC security, we can give security margins for the mitigation under the attacker model proposed in Section 3.1. As described in Section 3.1, the attacker needs to find a bitflip pattern that successfully performs a forgery attack on the MAC. On a system with our proposed mitigation, this is harder than in regular cryptographical attack scenarios, as the attacker only has partial knowledge on the input and no knowledge about the MAC.

In the following analyses, the MAC function are denoted as M_K , where K is the key of the specific MAC instance. Data words are labelled with $W, W', \dots$. Following the design of our mitigation, all MACs are 64 bit wide and all data words comprise 512 bit. We denote the set of all possible corrupted data words for a k -bit bitflip pattern as $\mathcal{W}^k$.

Given a 64 bit MAC, the overall number of possible tags is 2^{64} . Similarly, the number of possible 512 bit data words is 2^{512} . Following from these numbers is it obvious that some of the data words must map to the same MAC. This is not a problem as such MAC collisions are randomly distributed and can, in general, not be exploited by the attacker. Nevertheless, in some cases such collisions can lead to silent data corruption. Hence, we need to analyze this case separately. To quantify the protection against attacks that induce bitflips, we need to consider how likely it is that an k -bit bitflip pattern results in a data word whose MAC collides with the original data word. As the attacker model allows for bitflips in the data and the MAC, both scenarios need to be discussed.

Data-only errors

Assuming that the attacker induces bitflips purely in the data word, the security analysis is based on simple combinatoric observations. Let V denote the number of modified data words that can be generated when applying a k -bit bitmask on the n -bit data word. We

can compute V as

$$V(k) = \binom{512}{k} = \frac{512!}{k!(512-k)!}. \quad (6.1)$$

Note that Equation 6.1 gives the number of unique bitflip patterns without further consideration of flippable bits and the likelihood of multi-bit errors. The resulting value of V is the number of elements in $\mathcal{W}^k$ for a certain value of k .

Given that W denotes the original data word and W' a random data word that is computed by applying one of the k -bit bitmasks, the probability that such a modified data word maps to one of the MACs that are not equal to the MAC of the original data word can be given as

$$P(M_K(W) \neq M_K(W')) = \frac{2^{64} - 1}{2^{64}}. \quad (6.2)$$

For V possible modified data words, we construct the overall probability that none of the MACs of the modified words collide with the original MAC. We consider each combination as an independent event and assume that each combination is only selected once. Hence, each bitflip pattern is unique and maps to all available MACs with the same probability. The overall probability for finding no collision with the original MAC can be computed as the joint probability over all independent events. This leads to the equation

$$P(\text{no collision}) = \prod_{W' \in \mathcal{W}^k} \left(\frac{2^{64} - 1}{2^{64}} \right) = \left(\frac{2^{64} - 1}{2^{64}} \right)^{V(k)}. \quad (6.3)$$

With this formula, we can evaluate how many bitflip masks the attacker has to apply to produce a forgery for a fixed data word. Note that this formula does not consider collisions that occur between modified data words. Such collisions are irrelevant, as there is no possibility for the attacker to detect them.

Equation 6.1 yields the number of possible combinations that are achievable through a k -bit bitflip mask. With the number of combinations depending on the number of flipped bits, we can compute the probabilities that no collisions occur using Equation 6.3.

Figure 6.1 shows how the number of induced bitflips correlates with the probability of a MAC collision. In both subplots, the blue curve represents the probability that the attacker does not find a desired MAC collision to a pre-defined and fixed data word. Due to the size of the MAC, this probability stays almost constant until the attacker tries around 10^{16} unique data words.

Assuming that the attacker can induce a k -bit multi-bit error, we also need to assume that it is possible to induce a $k - 1$ -bit error. This indicates that an attacker is able to induce bitflips up to a certain number of flipped bits. Thus, the green curve depicts the probability for the scenario in which the attacker cumulatively induces bitflips up to k bit. As the number of combinations grows much faster with each additional bit than the sum of all previous combinations, this probability curve differs only slightly from the non-cumulative version. This indicates that even under the stronger attacker model, the MAC offers sufficient protection against the blind forgery attacks that the

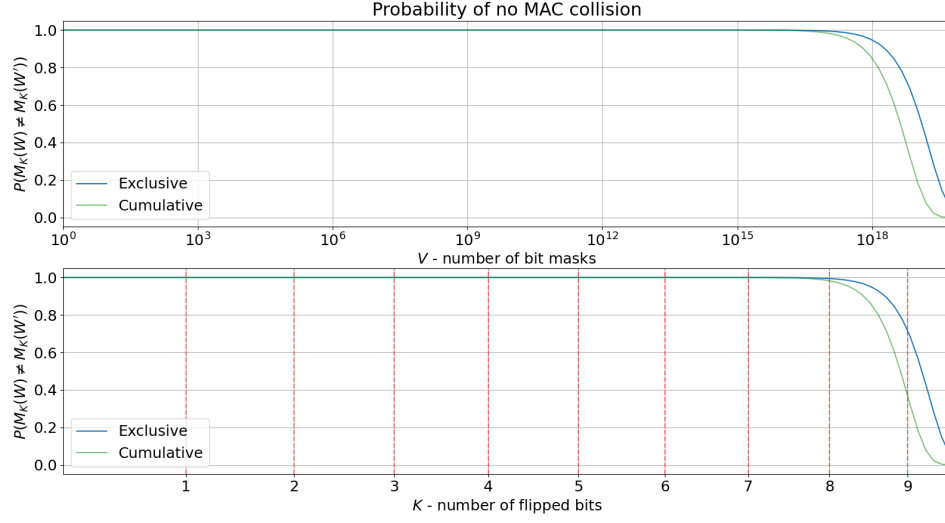


Figure 6.1: Influence of combinations and bitflips on the collision probability

attacker can mount. Specifically, the results suggest that the attacker would need to test all possible bitflip combinations up to all 8-bit errors to find a collision with a probability of approximately 3%.

By solving Equation 6.3 for V , we can estimate the number of unique bitflip patterns that an attacker has to try to reach a certain probability of success. The connection between the collision probability and the number of tested bit flip patterns is as follows:

$$P(\text{Collision}) = 1 - \left(\frac{2^{64} - 1}{2^{64}} \right)^{V(k)} \Leftrightarrow V(k) = \frac{\log(1 - P(\text{Collision}))}{\log\left(\frac{2^{64}-1}{2^{64}}\right)}. \quad (6.4)$$

Table 6.2 summarizes the results gained with Equation 6.4. Clearly, the attacker needs to try at least all 8-bit bitflip combinations on the 512-bit data word to have a slight chance of finding the correct collision. In the table, the second column shows the overall number of expected unique patterns that the attacker has to test. For all given probabilities, these numbers are sufficiently high to guarantee that the attacker will not be able to find the desired collision.

Mixed errors

According to our attacker model, it is likely that some of the MAC bits will also be corrupted by Rowhammer. Thus, we need to evaluate the reachable security levels under the hypothesis that some of the MAC bits are compromised. Given that the attacker can never gain read access to the MAC or the key, it is impossible to deliberately change bits in the MAC such that a collision is forced. Instead, the attacker would have to

Table 6.2: Correlation between collision probability and unique bitflip patterns

Collision Probability	Unique Patterns	Flipped Bits k
5%	$2^{59.71}$	$8 \leq k \leq 9$
10%	$2^{60.75}$	$8 \leq k \leq 9$
25%	$2^{62.20}$	$8 \leq k \leq 9$
50%	$2^{63.47}$	$9 \leq k \leq 10$
75%	$2^{64.47}$	$9 \leq k \leq 10$

blindly alter bits in the MAC and hope that the altered MAC is equal to the MAC of the corrupted data. To model this scenario, we define the k -bit neighborhood of MAC M as follows:

Given a MAC M , the k -bit neighborhood of M is a set $\mathcal{M}^k$ such that all MACs in $\mathcal{M}^k$ have a Hamming distance of exactly k bit to M . Note that, per definition, M itself is not part of this set.

This definition is similar to the set $\mathcal{W}^k$. For a successful attack, the attacker needs to alter the data and the stored MAC such that the computed MAC of the data is equal to the stored MAC. Under the assumption that the attacker compromises b bits of data and k bits of MAC, this is only possible if the computed MAC of the modified data is an element of $\mathcal{M}^k$ of the original MAC M . Similar to the number of possible corrupted data words, we can compute the number of elements in $\mathcal{M}^k$ depending on k as

$$|\mathcal{M}^k| = \frac{64!}{k!(64-k)!} \quad (6.5)$$

From Equation 6.5 it follows that the probability that a randomly chosen data word W' maps to one of the MACs in $\mathcal{M}$ can be given as

$$P(M_K(W') \in \mathcal{M}^k) = \frac{|\mathcal{M}^k|}{2^{64}} \quad (6.6)$$

For the attacker to succeed, the corrupted data word needs to map to the corrupted MAC. As Equation 6.6 gives the probability for a data word to map to any of the possible modified MACs, we can compute the probability that a corrupted data word W' maps to the exact MAC M' as

$$P(M_K(W') = M') = P(M_K(W') \in \mathcal{M}^k) \cdot \frac{1}{|\mathcal{M}^k|} = \frac{1}{2^{64}} \quad (6.7)$$

This result shows that the case of mixed errors is not advantageous for the attacker in the sense of a forgery attack. Even when the attacker flips bits in the MAC, the probability that the MAC matches the corrupted data is still equal to randomly guessing the MAC. For multiple corrupted data words, the probability scales linearly with the number of tested bitflip masks.

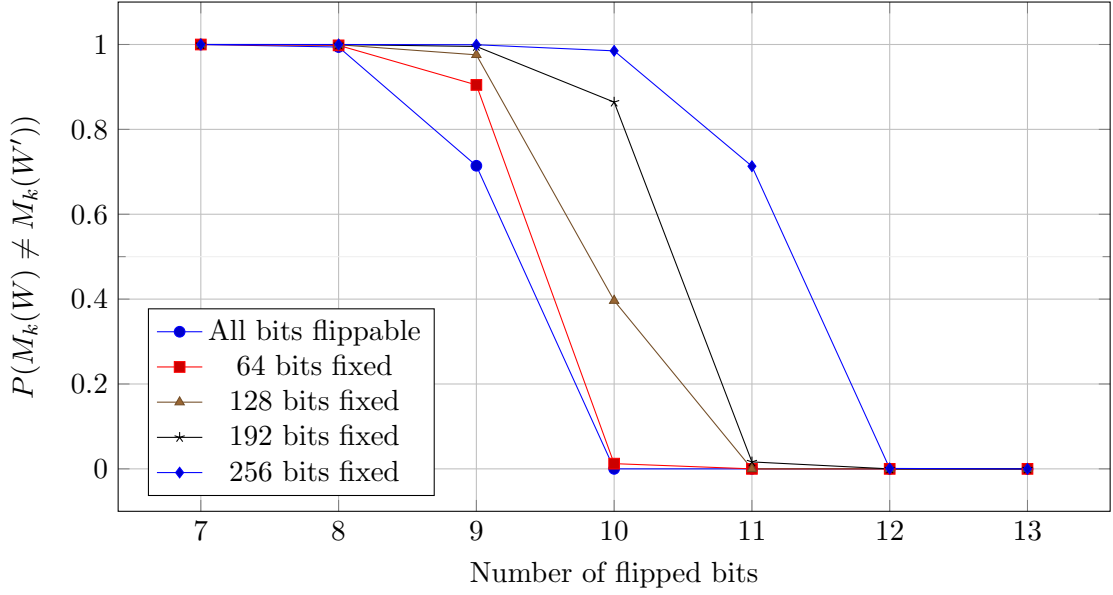


Figure 6.2: Non-affected bits increase the necessary attacker effort. In a realistic scenario, the attacker will not be able to try all possible bitflip combinations. Thus, the needed effort increases significantly.

The analysis done above is based on the assumption that the attacker can apply all unique bitflip masks to the data word. In reality, it might not be possible for the attacker to test all combinations that are theoretically reachable with a certain number of flipped bits. Usually, not all memory cells in a row are affected by Rowhammer. Thus, the number of reachable unique corrupted data words is smaller than the computed number of combinations. This fact greatly limits the capabilities of the attacker, as each non-flippable bit strongly reduces the number of reachable corrupted data words.

Figure 6.2 illustrates how row bits that are not affected by Rowhammer increase the security margin. If the number of unaffected bits increases, the attacker has to flip more bits to cause a collision. According to results of Mutlu et al. [MK20], only a small share of induced errors produces multi-bit errors. In most of the cases, Rowhammer attacks induce single or double bitflips into the victim rows. This indicates that increasing the number of flipped bits in a row is hard for the attacker and, thus, unaffected memory cells greatly increase the necessary effort.

As the number of unaffected bits strongly depends on the actual DRAM module under attack, basing the security on some assumption regarding this number is not an option. Instead, we can only give a lower bound on the security margin. This lower bound is exactly the computed effort from the analysis of data-only errors, as Equation 6.7 shows that bitflips in the MAC do not influence the security margin negatively.

6.2.1 Silent Data Corruption

During the correction step it might happen that the correction algorithm finds a data word that maps to the correct MAC but is not equal to the original data word. An attacker could mount an attack aiming to trigger exactly this effect, as silent data corruption greatly increases the number of corrupted bits. We want to analyze how probable this case is and if it is a feasible attack vector.

Assuming that the attacker induces a k -bit error, silent data corruption occurs if the correction algorithm finds a bitmask B that fulfills:

$$M_K(W' \oplus B) = M \quad (6.8)$$

where M is the stored MAC and B is not equal to the attackers bitflip pattern. We can base the analysis of this scenario on the results given in Figure 6.1. The cumulative probability for a fixed number of flipped bits is equivalent to the probability that we do not find an invalid correction. This indicates that it is highly unlikely that the correction algorithm will fail and correct the faulty data word back to an incorrect value.

Thus, we can conclude that silent data corruption is not a feasible attack vector for a malicious party. If the attacker can induce enough bit faults to cause the algorithm to miscorrect, the attacker would also be able to flip enough bits to bypass the detection mechanism in the first place. Also, performing this kind of attack is infeasible regarding the needed time. Each invocation of the correction algorithm takes time depending on the number of flipped bits. During this time, the attacker cannot proceed to flip bits, as the memory content will be overwritten once the correction is finished.

6.3 Time Complexity of the Error Correction

This section analyzes the correction procedure and gives best-, average- and worst-case estimations of correction times for different error scenarios.

The main factor influencing the time it takes to correct an error is the number of flipped bits. During the correction, the basic algorithm does not explicitly differentiate between data and MAC. Thus, we can compute the number of possible n -bit bitmasks for the given size constraints as given in Equation 6.9.

$$\# \text{Possible Bitmasks} = \frac{(512 + 64)!}{n! \cdot (512 + 64 - n)!} \quad (6.9)$$

Our proposed correction algorithm iterates over all bitmasks with a certain number of bits set before proceeding to the next greater Hamming distance. Starting with bitmasks of low Hamming weight exploits the fact that most bitflips are single- or double-bit errors [MK20].

Given that the attacker can induce an n -bit fault, the correction algorithm must test all bitmasks leading up to the correct n -bit pattern. In the worst case, the algorithm tests the correct bitmask as the last possible combination of the given Hamming distance. All preceding bitmasks are tested. Thus, the overall number of queried bitmasks is the sum

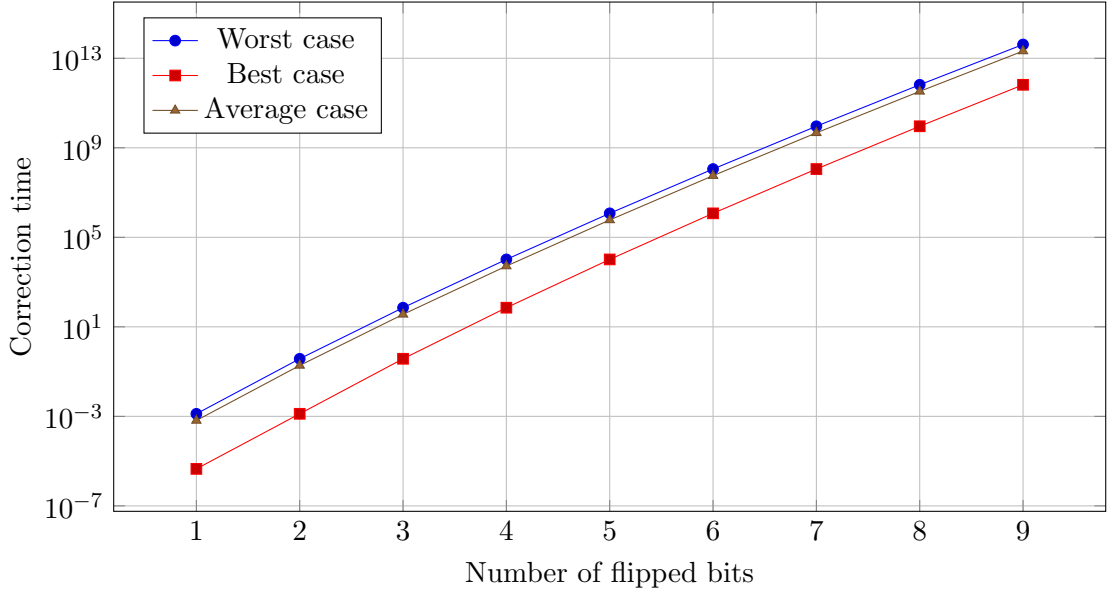


Figure 6.3: Influence of the bitflip count on the correction time

of all possible bitmasks of a lower degree. This iterative approach leads to a worst-case number of tries as given in Equation 6.10.

$$\# \text{Tested Bitmasks} = \sum_{k=0}^n \frac{(512 + 64)!}{k! \cdot (512 + 64 - k)!} \quad (6.10)$$

Note that we start at $k = 0$ to include the fact that the data might already be correct at the start of the correction handler in the case of parallel correction handling.

From the number of bitmasks, we can estimate the time needed for the correction loop by multiplying the number of tested bitmasks with the time that a single loop iteration takes. We measure the time for a loop iteration in the simulator, and thus it strongly depends on the system configuration. In the configuration of System 1, we measure an average latency between loop iterations of 225 CPU cycles using System 1. Under the given configuration, this evaluates to 225 ns for each loop iteration. Naturally, a higher CPU frequency can lead to even lower correction times.

Figure 6.3 shows the relation between the number of flipped bits and the expected time it takes to find a correction. In the worst case, we test all possible bitmasks and find the correct mask in the final query. The best-case occurs when the correct mask is the first mask of the correct Hamming distance. In the case of multi-bit errors, all masks of lower Hamming weight still need to be tested. Hence, the best-case performance for a n -bit error is only one query worse than the worst-case performance for a $n - 1$ -bit error. Assuming that bitflips are independently identically distributed, we can conclude that, on average, approximately half of all bitmasks of the correct Hamming distance need to

be tested to find the matching pattern. Thus, the average-case is twice as fast as the worst-case.

As seen in the figure, the time it takes to correct an error increases approximately exponentially with the number of flipped bits. While this might suggest a significant impact on performance, one needs to consider that this overhead is only relevant in the error case. As multi-bit errors are rare and even Rowhammer attacks seldomly induce more than two faults in the same row, we do not expect to see a considerable impact on system availability due to our mitigation.

6.4 Performance

Our mitigation is closely tied to the memory controller and DRAM. Thus, we need to evaluate how the mitigation will influence memory performance. During our evaluation, we consider two metrics that are of particular interest. Additional latency on memory accesses and memory overhead introduced by our mitigation should be kept to a minimum to avoid a negative impact on the system’s overall performance. The evaluation of these metrics is based on mathematical considerations and empirical results gained through simulation.

6.4.1 Memory Overhead

When protecting data integrity, redundancy is one of the principal concepts to check for data corruption. In our approach, the redundant data is the MAC computed over the protected data. Like in regular ECC-DRAM modules, we store the MAC on a separate, ninth, chip on the DRAM. Given that our mitigation protects each 512 bit data chunk with a 64 bit MAC, we can easily compute that for each chunk, we reach a memory overhead of 12.5%. This percentual overhead is equivalent to that of a regular SEC-DED code used by commodity ECC-DRAM modules. Due to the chosen MAC size, our mitigation can be deployed without any changes to the DRAM modules. Thus, it is backward-compatible with existing ECC-DRAM modules. In modern systems, reads and writes are always performed on cache-line granularity, which is 64 bytes. Hence, having a data word that is smaller or larger than a cache line would not fully utilize the way that memory accesses are handled.

6.4.2 Performance Impact

Due to the large number of memory accesses performed every second, memory latency significantly impacts system performance. Thus, we aim to eliminate any additional latencies our mitigation would introduce. The way the MAC is computed allows validation to occur as soon as the last chunk of data in a burst access is loaded. The critical word is loaded first, and the CPU can continue transient execution without waiting for the validation result. This design introduces only a little additional latency on memory accesses if no error occurs. Thus, our mitigation hardly impacts memory latency during

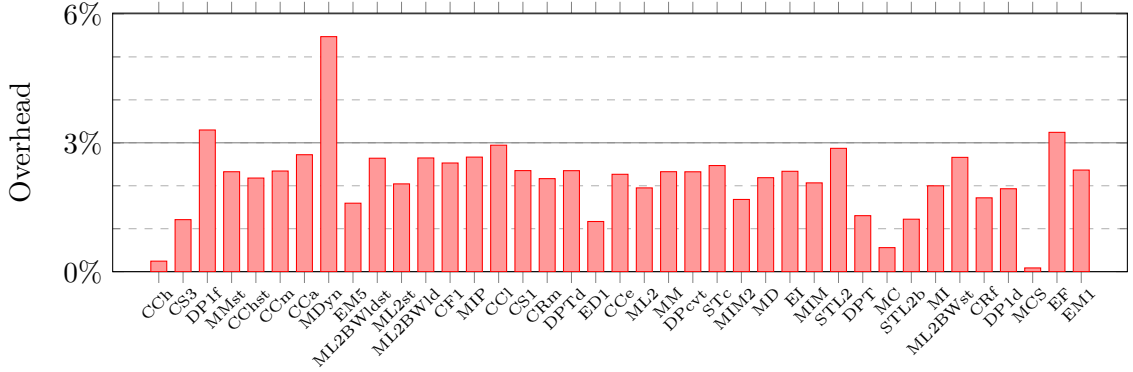


Figure 6.4: Results for the microbench benchmarks. We observe a peak overhead of 5 %. The average overhead is 2.17 %.

nominal operation. The slowdown for accesses to corrupted data depends on the number of bitflips in the corrupted block and is discussed further below in Section 6.3.

To evaluate the performance impact, we simulate different workloads with and without the mitigation in place. These simulations are done with System 2, as we want to include the effects that the MAC latency has on instruction reordering. We model each memory access such that the last access of a burst introduces a small additional overhead. This overhead represents the time between the last part of the burst and finishing the MAC validation. Note that this model does not consider transient execution, and hence an actual implementation of our design will result in even less performance overhead.

The evaluation is done with a subset of the PARSEC 2 benchmark suite [Bie+08] and a set of mini-benchmarks developed by Nowatzki et al. [Now+15]. Workloads are simulated in two configurations of System 2, one that implements our mitigation and one that does not. We switch between the two configurations by setting the MAC latency to 3.5 ns and 0 ns respectively. The normalized performance of the system without our mitigation is the reference performance, and we compare the simulation results against it to quantify the impact of our mitigation.

Figure 6.4 illustrates the results for the different benchmarks of the microbench suite. These test programs are small workloads that only run for a small number of CPU cycles. Memory-intensive test cases like MDyn show a more significant performance impact than benchmarks that use more advantageous access patterns. Overall, the average system performance impact for this set of benchmarks is around 2.17 % overhead. This performance is comparable to the performance impact of commodity ECC-DRAM modules.

Figure 6.5 shows the simulation results for workloads from the PARSEC benchmark suite. The average overhead drops significantly for larger workloads. On average, we detect an overhead of only 0.7 %. This result suggests that the influence of our

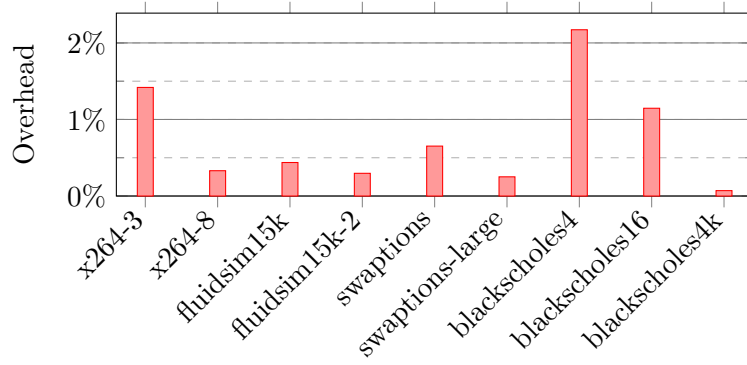


Figure 6.5: Results for the PARSEC benchmarks. Here, the maximum overhead is slightly over 2 %. The average overhead for the PARSEC benchmarks is only 0.7 %.

mitigation decreases with increasing workload. Especially in the case of the **blackscholes** benchmarks, one can see that with increasing complexity, the overhead decreases.

Note that the results presented in Figure 6.5 do not include all tests from the PARSEC benchmark suite. This is due to long simulation times for larger workloads.

From the above results, we can conclude that our mitigation only imposes a negligible performance overhead for a system with realistic workloads. Compared to the overhead of commodity ECC solutions, our mitigation has less impact on system performance. Thus, it is applicable to real-life systems, even for performance-critical applications. According to Kwon et al. [KSA14], ECC imposes a latency overhead of approximately 5 %. Compared to that, our mitigation only introduces an average latency overhead of 0.7 %. Thus, we outperform commodity ECC by a factor of 7.

Overall, our benchmark results show that the small overhead that the MAC computation introduces does not impact system performance significantly. Interestingly, for large workloads our mitigation performs even better than commodity ECC solutions.

Chapter 7

Conclusion

In this thesis, we presented a novel Rowhammer mitigation that simultaneously acts as integrity protection for DRAM modules. Due to the design of our countermeasure, the mitigation uses commodity ECC-DRAM modules. We proposed protecting DRAM data with cryptographic MAC functions stored in the additional ninth chip on the ECC-DRAM module. Furthermore, we shifted the error correction from hardware to software. This choice significantly increases the flexibility of our solution compared to commodity ECC implementations.

For our mitigation, several modifications to the current x86-64 architecture were necessary. We add a circuit to the memory controller that performs the MAC computation in hardware using parallel instances of a tweakable block cipher. Hardware-based MAC computation reduces additional latencies and, thus, increases performance. We introduced a new fault type that is triggered if the memory controller detects a mismatch between the computed MAC and the MAC stored on the DRAM. This fault triggers the execution of a novel fault handler that we added to the Linux kernel. By providing multiple instances of the fault handler and calling them in a round-robin fashion, we increase the stability of our mitigation as we account for bitflips in the error handler itself. We added a novel instruction to the AVX-512 instruction set extension that handles the MAC computation during the correction procedure.

We created a reference implementation of our architecture modifications in the gem5 simulator. In this implementation, we modified the memory controller such that it supports ECC-DRAM. Furthermore, we implemented the MAC computation using QARMA as a reference cipher due to its low latency. The memory controller modifications were implemented independently from the simulated CPU type. By extending the pre-existing implementation of burst accesses in DRAM modules, our implementation accurately simulates the behavior of the actual memory controller with the MAC circuit. As gem5 only supported a subset of the required vector instructions, we extended the instruction set and added the novel instruction used by the correction procedure. We implemented the novel fault type for the `AtomicSimpleCPU` and the `TimingSimpleCPU`, thus, allowing us to test the error handling with these two CPU models.

A basic correction procedure and the fault handler for the new fault type were implemented in the Linux kernel. The correction procedure was implemented such that it disables all other interrupts on the core that executes the correction code. In the reference implementation, we correct up to double bitflips through iteratively applied

bit-masks. We also added instrumentation instructions to the correction code to precisely measure the introduced overhead in the simulator.

We tested our modified system by comparing the cycle counts of different workloads to the measurements done on an unmodified system. The results suggested that the overhead introduced by our mitigation is in the range of commodity ECC implementations for small workloads. For larger workloads, our measurements show that our mitigation performs better than ECC, thus proving the applicability of our mitigation.

With our novel hardware-software co-design, we pave the way for even more sophisticated solutions.

With modification of the MAC, it is possible to include linear parity information. This would allow for a correction algorithm that is partially aware of the location of the flipped bits. Such a design would help to decrease the correction time.

The correction mechanism for single-bit errors could be implemented as a separate hardware circuit in the memory controller. Doing so would greatly increase performance in the case of simple bitflips.

Bibliography

- [AAA17] Misiker Tadesse Aga, Zelalem Birhanu Aweke, and Todd Austin. “When good protections go bad: Exploiting anti-DoS measures to accelerate Rowhammer attacks”. In: *HOST*. 2017.
- [Ava17] Roberto Avanzi. “The QARMA Block Cipher Family: Almost MDS Matrices Over Rings With Zero Divisors, Nearly Symmetric Even-Mansour Constructions With Non-Involutory Central Rounds, and Search Heuristics for Low-Latency S-Boxes”. In: *IACR Transactions on Symmetric Cryptology* 2017.1 (2017), pp. 4–44.
- [Awa+11] Manu Awasthi, David W. Nellans, Rajeev Balasubramonian, and Al Davis. “Prediction Based DRAM Row-Buffer Management in the Many-Core Era”. In: *PACT*. 2011.
- [Awe+16] Zelalem Birhanu Aweke, Salessawi Ferede Yitbarek, Rui Qiao, Reetuparna Das, Matthew Hicks, Yossi Oren, and Todd Austin. “ANVIL: Software-based protection against next-generation Rowhammer attacks”. In: *ACM SIGPLAN Notices* (2016).
- [Ban+88] SANJAY Banerjee, D Coleman, W Richardson, and A Shah. “Leakage mechanisms in the trench transistor DRAM cell”. In: *IEEE transactions on electron devices* 35.1 (1988), pp. 108–116.
- [BC05] Daniel P Bovet and Marco Cesati. *Understanding the Linux Kernel: from I/O ports to process management.* ” O’Reilly Media, Inc.”, 2005.
- [Bie+08] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. “The PARSEC benchmark suite: Characterization and architectural implications”. In: *Proceedings of the 17th international conference on Parallel architectures and compilation techniques*. 2008, pp. 72–81.
- [Bin+11] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. “The gem5 simulator”. In: *ACM SIGARCH computer architecture news* (2011).
- [BK12] Elaine Barker and John Kelsey. “NIST Special Publication 800-90A Recommendation for Random Number Generation Using Deterministic Random Bit Generators”. In: (2012).
- [BM16] Sarani Bhattacharya and Debdeep Mukhopadhyay. “Curious Case of Rowhammer: Flipping Secret Exponent Bits Using Timing Analysis”. In: *CHES*. 2016.

Bibliography

- [Bra+16] Ferdinand Brasser, Lucas Davi, David Gens, Christopher Liebchen, and Ahmad-Reza Sadeghi. “Can’t touch this: Practical and generic software-only defenses against rowhammer attacks”. In: *arXiv preprint arXiv:1611.08396* (2016).
- [BSS07] Ludger Borucki, Guenter Schindlbeck, and Charles Slayman. “Impact of DRAM process technology on neutron-induced soft errors”. In: *2007 IEEE International Integrated Reliability Workshop Final Report*. 2007, pp. 143–146. DOI: 10.1109/IRWS.2007.4469242.
- [Cha+87] TY Chan, J Chen, PK Ko, and C Hu. “The impact of gate-induced drain leakage current on MOSFET scaling”. In: *1987 International Electron Devices Meeting*. IEEE. 1987, pp. 718–721.
- [Cho+06] MH Cho, YI Kim, DS Woo, SW Kim, MS Shim, YJ Park, WS Lee, and BI Ryu. “Analysis of thermal variation of dram retention time”. In: *2006 IEEE International Reliability Physics Symposium Proceedings*. IEEE. 2006, pp. 433–436.
- [CJ10] Elliott Cooper-Balis and Bruce Jacob. “Fine-grained activation for power reduction in DRAM”. In: *IEEE Micro* 30.3 (2010), pp. 34–47.
- [Coj+19] Lucian Cojocar, Kaveh Razavi, Cristiano Giuffrida, and Herbert Bos. “Exploiting correcting codes: On the effectiveness of ecc memory against rowhammer attacks”. In: *S&P*. 2019.
- [Fri+20] Pietro Frigo, Emanuele Vannacci, Hasan Hassan, Victor van der Veen, Onur Mutlu, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. “TRRespass: Exploiting the Many Sides of Target Row Refresh”. In: *S&P*. 2020.
- [Ger+05] Eric Gerritsen, Nicolas Emonet, Christian Caillat, Nicolas Jourdan, Marc Piazza, David Fraboulet, Bruce Boeck, Audrey Berthelot, Steven Smith, and Pascale Mazoyer. “Evolution of materials technology for stacked-capacitors in 65 nm embedded-DRAM”. In: *Solid-State Electronics* 49.11 (2005), pp. 1767–1775.
- [GMM16] Daniel Gruss, Clémentine Maurice, and Stefan Mangard. “Rowhammer.js: A Remote Software-Induced Fault Attack in JavaScript”. In: *DIMVA*. 2016.
- [Gru+18] Daniel Gruss, Moritz Lipp, Michael Schwarz, Daniel Genkin, Jonas Juffinger, Sioli O’Connell, Wolfgang Schoechl, and Yuval Yarom. “Another Flip in the Wall of Rowhammer Defenses”. In: *S&P*. 2018.
- [Ham50] Richard W Hamming. “Error detecting and error correcting codes”. In: *The Bell system technical journal* (1950).
- [HF15] Nishad Herath and Anders Fogh. “These are Not Your Grand Daddys CPU Performance Counters – CPU Hardware Performance Counters for Security”. In: *Black Hat Briefings*. 2015.
- [HS10] Ruirui Huang and G. Edward Suh. “IVEC: Off-Chip Memory Integrity Protection for Both Security and Reliability”. In: *ISCA*. 2010.

- [Huh+95] Yoonjong Huh, Dooyoung Yang, Hyungsoon Shin, and Yungkwon Sung. “Hot-carrier-induced circuit degradation in actual DRAM”. In: *Proceedings of 1995 IEEE International Reliability Physics Symposium*. IEEE. 1995, pp. 72–75.
- [IES18] Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. “MASCAT: Preventing microarchitectural attacks before distribution”. In: *CODASPY*. 2018.
- [Int12] Intel. *Intel Architecture Instruction Set Extensions Programming Reference*. 2012.
- [Int17] Intel. *Single- and Multichannel Memory Modes*. Accessed: 2021. 2017. URL: <https://www.intel.com/content/www/us/en/support/articles/000005657/boards-and-kits.html>.
- [Int19a] Intel. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. 2019.
- [Int19b] Intel. *Intel 64 and IA-32 Architectures Software Developer’s Manual Volume 2 (2A, 2B & 2C): Instruction Set Reference, A-Z*. 2019.
- [Int21a] Intel. *11th Generation Intel Core Processor Family, Volume 1*. 2021. URL: <https://cdrdv2.intel.com/v1/dl/getContent/634648>.
- [Int21b] Intel. *Intel® 64 and IA-32 Architectures Software Developer’s Manual Volume 2 - Instruction Set Reference*. Apr. 2021. URL: <https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-instruction-set-reference-manual-325383.pdf>.
- [JED13] JEDEC Solid State Technology Association. *Low Power Double Data Rate 3*. 2013. URL: <http://www.jedec.org/standards-documents/docs/jesd209-4a>.
- [JED17] JEDEC Solid State Technology Association. *Low Power Double Data Rate 4*. 2017. URL: <http://www.jedec.org/standards-documents/docs/jesd209-4b>.
- [JED20] JEDEC Solid State Technology Association. *JESD79-5 DDR5 SDRAM*. 2020. URL: <https://www.jedec.org/standards-documents/docs/jesd79-5>.
- [Jun+16] Matthias Jung, Carl C Rheinländer, Christian Weis, and Norbert Wehn. “Reverse engineering of DRAMs: Row hammer with crosshair”. In: *International Symposium on Memory Systems*. 2016.
- [JWN10] Bruce Jacob, David Wang, and Spencer Ng. *Memory systems: cache, DRAM, disk*. Morgan Kaufmann, 2010.
- [KBC97] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. *HMAC: Keyed-hashing for message authentication*. 1997.

Bibliography

- [Kim+01] Young Pil Kim, Young Wook Park, Joo Tae Moon, and SU Kim. “Reliability degradation of high density DRAM cell transistor junction leakage current induced by band-to-defect tunneling under the off-state bias-temperature stress”. In: *2001 IEEE International Reliability Physics Symposium Proceedings. 39th Annual (Cat. No. 00CH37167)*. IEEE. 2001, pp. 1–6.
- [Kim+14] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors”. In: *ISCA*. 2014.
- [Kim+20] Jeremie S Kim, Minesh Patel, A Giray Yağlıkçı, Hasan Hassan, Roknoddin Azizi, Lois Orosa, and Onur Mutlu. “Revisiting rowhammer: An experimental analysis of modern dram devices and mitigation techniques”. In: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE. 2020, pp. 638–651.
- [Kon+18] Radhesh Krishnan Konoth, Marco Oliverio, Andrei Tatar, Dennis Andriesse, Herbert Bos, Cristiano Giuffrida, and Kaveh Razavi. “ZebRAM: Comprehensive and Compatible Software Protection Against Rowhammer Attacks”. In: *USENIX OSDI*. 2018.
- [KSA14] Sanghyuk Kwon, Young Hoon Son, and Jung Ho Ahn. “Understanding ddr4 in pursuit of in-dram ecc”. In: *2014 International SoC Design Conference (ISOCC)*. IEEE. 2014, pp. 276–277.
- [Lip+20] Moritz Lipp, Misiker Tadesse Aga, Michael Schwarz, Daniel Gruss, Clémentine Maurice, Lukas Raab, and Lukas Lamster. “Nethammer: Inducing Rowhammer Faults through Network Requests”. In: *SILM Workshop*. 2020.
- [Low20] Jason Lowe-Power. *gem5 Documentation*. Accessed: 2021. 2020. URL: <https://www.gem5.org/documentation/>.
- [Luc04] Stefan Lucks. *Design Principles for Iterated Hash Functions*. Cryptology ePrint Archive, Report 2004/253. <https://ia.cr/2004/253>. 2004.
- [MK20] Onur Mutlu and Jeremie S. Kim. “RowHammer: A Retrospective”. In: *IEEE TCAD* (2020).
- [Nor96] Eugene Normand. “Single event upset at ground level”. In: *IEEE transactions on Nuclear Science* 43.6 (1996), pp. 2742–2750.
- [Now+15] Tony Nowatzki, Jaikrishnan Menon, Chen-Han Ho, and Karthikeyan Sankaralingam. “Architectural simulators considered harmful”. In: *IEEE Micro* 35.6 (2015), pp. 4–12.
- [Oh+11] Byoungchan Oh, Heung-Jae Cho, Heesang Kim, Younghwan Son, Taewook Kang, Sunyoung Park, Seunghyun Jang, Jong-Ho Lee, and Hyungcheol Shin. “Characterization of an oxide trap leading to random telegraph noise in gate-induced drain leakage current of DRAM cell transistors”. In: *IEEE transactions on electron devices* 58.6 (2011), pp. 1741–1747.

- [Pes+16] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. “DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks”. In: *USENIX Security Symposium*. 2016.
- [Raz+16] Kaveh Razavi, Ben Gras, Erik Bosman, Bart Preneel, Cristiano Giuffrida, and Herbert Bos. “Flip Feng Shui: Hammering a Needle in the Software Stack”. In: *USENIX Security Symposium*. 2016.
- [Rid+21] Finn de Ridder, Pietro Frigo, Emanuele Vannacci, Herbert Bos, Cristiano Giuffrida, and Kaveh Razavi. “SMASH: Synchronized Many-sided Rowhammer Attacks From JavaScript”. In: *USENIX Security Symposium*. 2021.
- [Rix+00] Scott Rixner, William J Dally, Ujval J Kapasi, Peter Mattson, and John D Owens. “Memory access scheduling”. In: *ACM SIGARCH Computer Architecture News* 28.2 (2000), pp. 128–138.
- [RK19] R Rooney and N Koyle. “Micron DDR5 SDRAM: New Features”. In: *Micron Technology Inc., Tech. Rep* (2019).
- [RPL92] Phillip J Restle, JW Park, and Brian F Lloyd. “DRAM variable retention time”. In: *IEDM Tech. Dig* (1992), pp. 807–810.
- [Sai+18] Gururaj Saileshwar, Prashant J Nair, Prakash Ramrakhiani, Wendy Elsasser, and Moinuddin K Qureshi. “Synergy: Rethinking secure-memory design for error-correcting memories”. In: *HPCA*. 2018.
- [Sal21] Salman Qazi, Yoongu Kim, Nicolas Boichat, Eric Shiu, Mattias Nissler. *Introducing Half-Double: New hammering technique for DRAM Rowhammer bug*. 2021. URL: <https://security.googleblog.com/2021/05/introducing-half-double-new-hammering.html>.
- [Sam14] Samsung. *DDR4 SDRAM Specification Rev 1.1*. Whitepaper. Accessed: 2021. 2014. URL: https://www.samsung.com/semiconductor/global.semi/file/resource/2017/11/DDR4_Device_Operations_Rev11_Oct_14-0.pdf.
- [SD15] Mark Seaborn and Thomas Dullien. “Exploiting the DRAM Rowhammer bug to gain kernel privileges”. In: *Black Hat Briefings*. 2015.
- [Sta03] William Stallings. *Computer organization and architecture: designing for performance*. Pearson Education India, 2003.
- [Tat+18a] Andrei Tatar, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. “Defeating software mitigations against rowhammer: a surgical precision hammer”. In: *RAID*. 2018.
- [Tat+18b] Andrei Tatar, Radhesh Krishnan, Elias Athanasopoulos, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. “Throwhammer: Rowhammer Attacks over the Network and Defenses”. In: *USENIX ATC*. 2018.

Bibliography

- [Vee+16] Victor van der Veen, Yanick Fratantonio, Martina Lindorfer, Daniel Gruss, Cl  mentine Maurice, Giovanni Vigna, Herbert Bos, Kaveh Razavi, and Cristiano Giuffrida. “Drammer: Deterministic Rowhammer Attacks on Mobile Platforms”. In: *CCS*. 2016.
- [Wan20] Zhengrong Wang. *Bring AVX Support to Gem5*. 2020. URL: <https://seanzw.github.io/posts/gem5-avx/> (visited on 12/2020).
- [Wei+19] Zane Weissman, Thore Tiemann, Daniel Moghimi, Evan Custodio, Thomas Eisenbarth, and Berk Sunar. “JackHammer: Efficient Rowhammer on Heterogeneous FPGA-CPU Platforms”. In: *arXiv:1912.11523* (2019).
- [WLB21] Andrew J Walker, Sungkwon Lee, and Dafna Beery. “On DRAM Rowhammer and the Physics of Insecurity”. In: *IEEE Transactions on Electron Devices* (2021).
- [Ya  +21] A Giray Ya  lik  i, Minesh Patel, Jeremie S Kim, Roknoddin Azizi, Ataberk Olgun, Lois Orosa, Hasan Hassan, Jisung Park, Konstantinos Kanellopoulos, Taha Shahroodi, et al. “BlockHammer: Preventing RowHammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows”. In: *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE. 2021, pp. 345–358.
- [Yan+87] David S Yaney, Chih-Yuan Lu, Ross A Kohler, Michael J Kelly, and James T Nelson. “A meta-stable leakage phenomenon in DRAM charge storage-Variable hold time”. In: *1987 International Electron Devices Meeting*. IEEE. 1987, pp. 336–339.
- [Zha+19] Zhi Zhang, Yueqiang Cheng, Dongxi Liu, Surya Nepal, and Zhi Wang. “TeleHammer: Cross-Privilege-Boundary Rowhammer through Implicit Accesses”. In: *arXiv:1912.03076* (2019).
- [Zha+20] Zhi Zhang, Yueqiang Cheng, Dongxi Liu, Surya Nepal, Zhi Wang, and Yuval Yarom. “PThammer: Cross-User-Kernel-Boundary Rowhammer through Implicit Accesses”. In: *MICRO*. IEEE, 2020, pp. 28–41.

List of Figures

2.1	DRAM Cell Structure	7
2.2	DRAM Bank Structure	8
2.3	DRAM Read Timing Diagram	9
2.4	Rowhammer Patterns	15
2.5	CPU Core Structure	22
2.6	Intel 64 Instruction Format	25
2.7	Gate Descriptor Data	29
2.8	gem5 System Structure	31
2.9	Merkle-Damgård iterated hash function	36
2.10	Schematic CBC-MAC construction	37
3.1	Mitigation Overview	41
3.2	Structure of the MAC used to protect data integrity	43
6.1	Influence of combinations and bitflips on the collision probability	58
6.2	Attacker effort depending on fixed bits	60
6.3	Influence of the bitflip count on the correction time	62
6.4	Microbench Results	64
6.5	PARSEC Results	65

List of Tables

2.1	Memory overhead for different SEC and SEC-DED codes	20
2.2	Content of the per-CPU IDT according to the Linux kernel sources	29
6.1	System configuration of the benchmark systems	56
6.2	Correlation between collision probability and unique bitflip patterns . . .	59